GOOGLE | ALL SOURCES | TOPIC 9 OF 10 | LESS COMMON TOPIC

Greedy and Math

13 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

8	5
HARD	MEDIUM

How to use

- Questions are Hard first, then Medium, then Easy, newest last-asked date first.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Greedy and Math | 13 questions | tap a title to jump to it

H	1. Check Digits and Combine Ranges	**PracHub**	**2026-03-29**
H	2. Find secret word with match-count feedback	**PracHub**	**2026-02-11**
H	3. Wordle-Style 5-Letter Minimum-Guess Strategy	**1Point3Acres**	**2025-12-15**
H	4. Shuffle array using random integer API	**PracHub**	**2025-12-08**
H	5. Find maximum value in huge text file	**PracHub**	**2025-12-08**
H	6. Find All Bad Pairs Using runTest Oracle	**1Point3Acres**	**2025-11-27**
H	7. Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whos...		**Exponent**
H	8. Implement a function to justify a list of words within a given width.		**Exponent**
M	9. Find the Largest Monotone Increasing Number	**PracHub**	**2026-07-04**
M	10. Min boards to cover roof holes	**PracHub**	**2026-02-12**
M	11. Choose optimal guesses for green-only Wordle	**PracHub**	**2025-10-14**
M	12. Write SQL code to publish the Fibonacci series.		**Exponent**
M	13. Trip Expense Settlement		**HackTheRounds**

STUDY HERE: topic 9, Greedy and Math

HARD | PracHub | Technical Screen | 1 of 13

★ TOP 75: rank 4 in the Medium and Hard priority list

★ Check Digits and Combine Ranges

POSTED 2026-03-29 | 205 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 73.6** **30 DAYS 63.4** **PRIORITY 2/228**

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #9 Palindrome Number, LeetCode #56 Merge Intervals, LeetCode #228 Summary Ranges

LeetCode match: LC 9 Palindrome Number + LC 56 Merge Intervals / LC 228 Summary Ranges

Posted on PracHub: 2026-03-29 (updated 2026-05-19)

Original: https://prachub.com/coding-questions/check-digits-and-combine-ranges

QUESTION

You are asked to solve two independent coding tasks.

Task 1:

Check Whether an Integer Reads the Same Backward Given an integer `x`, return `true` if its decimal representation reads the same from left to right and from right to left; otherwise return `false`. Requirements and edge cases:

- Negative integers should return `false` because of the leading minus sign.
- You may assume `x` fits in a signed 32-bit integer.
- Try to solve it without converting the integer to a string. Example:

```
Input: x = 121
Output: true
```

```
Input: x = -121
Output: false
```

```
Input: x = 10
Output: false
```

Task 2:

Combine Overlapping Ranges Given an array of intervals `intervals`, where each interval is represented as `[start, end]`, combine all overlapping intervals and return an array of non-overlapping intervals that covers exactly the same ranges. Requirements and edge cases:

- Intervals may be provided in any order.
- Two intervals overlap if they share at least one point. For example, `[1, 4]` and `[4, 5]` should be combined into `[1, 5]`.

QUESTION

- Return the intervals in increasing order by start value. Example:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
```

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
```

Part 1: Check Whether an Integer Reads the Same BackwardGiven an integer x, return True if its decimal representation reads the same from left to right and from right to left. Otherwise, return False. Negative integers are not palindromes because of the leading minus sign. Try to solve the problem using arithmetic operations instead of converting the integer to a string.
Constraints-2^31 <= x <= 2^31 - 1Negative integers must return FalseAn arithmetic-based solution is preferred over string conversion

EXAMPLES

```
Input: 121
Expected Output: True
```

Explanation: 121 reads the same forward and backward.

```
Input: -121
Expected Output: False
```

Explanation: Negative numbers are not palindromes.

Hints

A negative number can never be a palindrome because the minus sign only appears on the left.Try reversing only the second half of the digits instead of the whole number.
Part 2: Combine Overlapping RangesGiven a list of intervals, where each interval is represented as [start, end], merge all overlapping intervals and return a new list of non-overlapping intervals that covers exactly the same ranges. Intervals may appear in any order. Two intervals overlap if they share at least one point, so [1, 4] and [4, 5] should be merged into [1, 5]. Return the result sorted by starting value.
Constraints
0 <= len(intervals) <= 10^4-10^9 <= start <= end <= 10^9Intervals may be unsorted
Examples
Input: [[1, 3], [2, 6], [8, 10], [15, 18]]
Expected Output: [[1, 6], [8, 10], [15, 18]]
Explanation: The first two intervals overlap and merge into [1, 6].
Input: [[1, 4], [4, 5]]
Expected Output: [[1, 5]]
Explanation: The intervals share the point 4, so they overlap.
Hints

Sort the intervals by their start value before trying to merge them.Keep track of the last interval in the output. If the next interval overlaps, extend the end; otherwise, start a new interval.

APPROACH

Task 1: reverse half the digits and compare (no string, handle negatives and trailing zeros). Task 2: sort ranges and merge or combine consecutive ones in one pass.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two independent tasks. For the palindrome: negatives are false, no string conversion, 32-bit input. For the ranges: ask whether touching ranges such as [1, 4] and [4, 5] merge (the statement says they do) and whether adjacent integer ranges like [1, 2] and [3, 4] should also merge.

BRUTE FORCE TO SAY OUT LOUD

Palindrome: convert to a string and compare with its reverse. Ranges: compare every pair repeatedly until nothing merges, O(n^2).

OPTIMAL IDEA

Palindrome: return false for negatives and for any positive number ending in 0; then build a reversed number from the low digits while x > rev, and compare x with rev or rev // 10 (to skip the middle digit). Only half the digits are processed, so overflow cannot happen. Ranges: sort by start, then sweep, extending the last merged range when the next start is at most its end, else append. O(n log n).

DRY-RUN THESE CASES

121 true, -121 false, 10 false, 0 true, 1221, 12321. Ranges: [[1,3],[2,6],[8,10],[15,18]] gives [[1,6],[8,10],[15,18]]; [[1,4],[4,5]] gives [[1,5]]; unsorted input; one range; a range fully inside another.

LIKELY FOLLOW-UPS

Palindrome in another base, or palindrome linked list. Ranges: insert a new interval into a sorted list (LeetCode 57), summary ranges of sorted integers (LeetCode 228).

PITFALLS

Reversing the whole number and overflowing. Forgetting to sort before merging. Using strict less-than when touching ranges must merge.

HOW TO THINK ABOUT IT

Task 1: a number is a palindrome if its second half reversed equals its first half, so reverse only half the digits (stop when the reversed part catches up) and avoid overflow. Negatives and numbers ending in 0 are never palindromes. Task 2: after sorting by start, an overlap can only be with the last merged range, so one pass with a 'current' range is enough.

Basis: Two independent tasks: LeetCode 9 (Palindrome Number, no string conversion) and LeetCode 56 (Merge Intervals; ranges sharing a point merge).

BRUTE FORCE · Task 1: convert to a string and compare with its reverse

BRUTE FORCE · PSEUDO-CODE

```
def is_palindrome(x):
    s = str(x)
    return x >= 0 and s == s[::-1]
```

TIME	SPACE
O(d)	O(d)

WHY IT IS SLOW

O(d) time but it builds strings, which the question forbids. Reversing only the second half of the digits arithmetically (x % 10, x // 10) uses O(1) space.

BRUTE FORCE · Task 2: merge any two overlapping ranges until none overlap

BRUTE FORCE · PSEUDO-CODE

```
def combine(ranges):
    ranges = list(ranges)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(ranges, 2):
            if overlaps(a, b):
                ranges.remove(a)
                ranges.remove(b)
                ranges.append(union(a, b))
                merged = True
                break
    return sorted(ranges)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION · Task 1: reverse only the second half arithmetically

TIME	SPACE
O(d)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task 2: sort by start, then one sweep

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Adjacent integer ranges ([1, 2] and [3, 4]) should merge too.

Change the overlap test to s <= out[-1][1] + 1 (integers only). For real-valued ranges that is wrong, so say which you assume.

FOLLOW-UP 2 Summarise a sorted list of integers as ranges (LeetCode 228).

One pass keeping the current range and extending it while the next number is exactly one more; the same 'extend or start new' structure as the merge.

FOLLOW-UP 3 Palindrome in another base, or as a linked list.

Another base replaces 10 by the base in % and //. For a linked list find the middle with slow and fast pointers, reverse the second half in place and compare.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check Digits and Combine Ranges
#
# You are asked to solve two independent coding tasks. ###
# Task 1: Check Whether an Integer Reads the Same Backward
# Given an integer x, return true if its decimal
# representation reads the same from left to right and
# from right to left; otherwise return false. Requirements
# and edge cases: - Negative integers should return false
# because of the leading minus sign. - You may assume x
# fits in a signed 32-bit integer. - Try to solve it
# without converting the integer to a string. Example:
# text Input: x = 121 Output: true text Input: x = -121
# Output: false text Input: x = 10 Output: false ### Task
# 2: Combine Overlapping Ranges Given an array of
# intervals intervals, where each interval is represented
# as [start, end], combine all overlapping intervals and
# return an array of non-overlapping intervals that covers
# exactly the same ranges.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Adjacent integer ranges ([1, 2] and [3, 4]) should
# merge too.
# 2. Summarise a sorted list of integers as ranges
# (LeetCode 228).
# 3. Palindrome in another base, or as a linked list.
#
# INTERVIEW NOTES
# Task 1: a number is a palindrome if its second half
# reversed equals its first half, so reverse only half the
# digits (stop when the reversed part catches up) and
# avoid overflow. Negatives and numbers ending in 0 are
# never palindromes. Task 2: after sorting by start, an
# overlap can only be with the last merged range, so one
# pass with a 'current' range is enough.
#
# BRUTE FORCE (say it first): Task 1: convert to a string
# and compare with its reverse
# str(x) == str(x)[::-1] after rejecting negatives.
# Correct and short; the question asks to avoid the
# string. Time O(d). Space O(d).
```

```
# BRUTE FORCE (say it first): Task 1: convert to a string
# and compare with its reverse | time O(d)
def palindrome_number_string(x):
    return x >= 0 and str(x) == str(x)[::-1]


# BASE QUESTION (optimal): reverse only the second half
# arithmetically
# Idea: Reject negatives and positive numbers ending in 0.
def is_palindrome(x, base=10):
    # no string: build the reversed second half
    if x < 0 or (x % base == 0 and x != 0):
        return False
    rev = 0
    # stop when the reversed half catches up with the rest
    while x > rev:
        rev = rev * base + x % base
        x //= base
    # an odd number of digits has a middle digit to drop
    return x == rev or x == rev // base


# TASK 2: combine ranges
def merge_ranges(intervals, adjacent=False):
    out = []
    # sort by start so overlaps are neighbours
    for s, e in sorted(intervals):
        # sharing one point counts as overlap: [1,4] +
        # [4,5]
        gap = 1 if adjacent else 0
        if out and s <= out[-1][1] + gap:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out


# FOLLOW-UP 2: Summarise a sorted list of integers as
# ranges (LeetCode 228).
# Answer: One pass keeping the current range and extending
# it while the next number is exactly one more; the same
# 'extend or start new' structure as the merge.
def summarize_ranges(nums):
    # LeetCode 228: a sorted list of ints as ranges
    out = []
    for x in nums:
        if out and x == out[-1][1] + 1:
            # exactly one more: extend the current range
            out[-1][1] = x
        else:
            out.append([x, x])
    return out


if __name__ == "__main__":
    assert is_palindrome(121) and is_palindrome(0)
    assert not is_palindrome(-121)
    assert not is_palindrome(10)
    assert is_palindrome(1221) and not is_palindrome(123)
    # binary 101 is 5; binary 110 is 6
    assert is_palindrome(5, 2) and not is_palindrome(6, 2)
    ranges = [[1, 3], [2, 6], [8, 10], [15, 18]]
    merged = merge_ranges(ranges)
    assert merged == [[1, 6], [8, 10], [15, 18]]
    assert merge_ranges([[1, 4], [4, 5]]) == [[1, 5]]
    # any input order, and an empty list
    shuffled = merge_ranges([[8, 10], [1, 3]])
    assert shuffled == [[1, 3], [8, 10]]
    assert merge_ranges([]) == []
```

```
    # adjacent integer ranges merge only when asked
    touching = [[1, 2], [3, 4]]
    assert merge_ranges(touching) == touching
    assert merge_ranges(touching, True) == [[1, 4]]
    assert summarize_ranges([0, 1, 2, 4, 5, 7]) == [
        [0, 2], [4, 5], [7, 7]]
```

FUNCTION AND TEST CASES

Signature

```
def solution(x: int) -> bool:
SAMPLE INPUT
121
```

Test cases

Input	Expected
121	True
-121	False

STUDY HERE: topic 9, Greedy and Math

HARD | IN APP LC 843 | PracHub | Technical Screen | 2 of 13

Find secret word with match-count feedback

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TYPE ALL TIME 42.2 30 DAYS 0 PRIORITY 84/228

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #843 Guess the Word

LeetCode match: LC 843 Guess the Word

Posted on PracHub: 2026-02-11 (updated 2026-04-30)

Original: https://prachub.com/coding-questions/find-secret-word-with-match-count-feedback

QUESTION

Problem

You are given a list of **unique** lowercase words, all of the same length (e.g., length = 6). One of these words is a **secret** word. You can interact with a black-box API:

- `matchCount(guess) -> int`: when you submit a word `guess` from the list, the API returns the number of positions `i` such that `guess[i] == secret[i]` (i.e., exact character match at the same index). You are allowed to call `matchCount`

at most 10 times.

Task

Design/implement a function (e.g., `findSecretWord(words)`) that uses the API to identify the secret word within the guess limit.

Notes / Constraints

- `words` contains between 1 and 100 words.
- All words have equal length `L` (commonly `L = 6`).
- Every guess must be a word from `words`.
- After each API response, you may adapt your next guess.

What to return

Depending on the interface, you may either: - return the identified secret word, or - stop once `matchCount(guess) == L` (found it). (Your interviewer may treat this as an interactive problem; focus on strategy + correctness under the 10-guess constraint.)

You are given a list of unique lowercase words, and every word has the same length `L`. Exactly one word in the list is the secret word. In the original interview version, you interact with a black-box API:

- `matchCount(guess) -> int`

QUESTION

- It returns how many positions `i` satisfy `guess[i] == secret[i]` You may call this API at most 10 times, and every guess must be a word from the given list. In this non-interactive version, implement `solution(words, secret)` and simulate the API inside your function. The parameter `secret` is provided only so the judge can verify your strategy. Your function should repeatedly make guesses, use the returned match counts to eliminate impossible candidates, and return the discovered secret word. If your strategy does not find the secret within 10 guesses, return an empty string `""`.

Constraints`1 <= len(words) <= 1001 <= len(words[i]) <= 10`All words are unique, lowercase, and have the same length `Lsecret` is guaranteed to be one of the words in `words`Your algorithm may use at most 10 simulated `matchCount` callsTest data is designed so that a correct elimination strategy can identify the secret within the limit

EXAMPLES

```
Input: (["acckzz", "ccbazz", "eiowzz", "abcczz"],
    "acckzz")
```

Expected Output: 'acckzz'Explanation: The secret is already one of the candidates. Using match-count feedback, the candidate set shrinks until `acckzz` is identified.

```
Input: (["secret"], "secret")
```

Expected Output: 'secret'Explanation: With only one word in the list, the first allowed guess must be the secret.

Hints

After a guess returns `k`, any valid remaining candidate must also have exactly `k` matching positions with that guess.A strong strategy is to choose the next guess that splits the remaining candidates as evenly as possible, minimizing the size of the largest feedback bucket.

APPROACH

Pick a guess, read matchCount, keep only candidates with the same match count against that guess. Choosing the guess that minimizes the worst-case remaining set (minimax) keeps queries low.

HOW TO THINK ABOUT IT

Each guess returns a number that every remaining candidate must agree with: a candidate can still be the secret only if it has the same match count with the guess as the secret does. So after each guess, filter the candidate list. To shrink fast, guess a word that splits the candidates evenly by match count (minimise the largest group); that is what keeps you inside 10 guesses.

Basis: LeetCode 843, Guess the Word (match count = positions with the same letter).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words):
    candidates = list(words)
    while True:
        guess = candidates[0]
        m = matchCount(guess)
        if m == len(guess):
            return guess
        candidates = [w for w in candidates
                      if matches(w, guess) == m]
```

TIME	SPACE
O(guesses * W * L)	O(W)

WHY IT IS SLOW

A poor guess may remove very few candidates, so there is no guarantee of finishing in 10 guesses. Guessing the word whose largest match-count group is smallest (minimax) keeps the list shrinking fast.

OPTIMAL SOLUTION · Guess the word that minimises the size of its largest match-count group

TIME	SPACE
O(guesses * W^2 * L)	O(W)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Show how the candidate list shrinks.

After a guess with answer m, only words with the same match count against the guess survive (the secret and the candidate agree on every comparison). Printing the sizes round by round shows why a balanced guess finishes in a few rounds.

FOLLOW-UP 2 Guess at random instead of the minimax choice.

Expected to finish in far fewer than W guesses, but not guaranteed within 10 on a hard list. The minimax choice is the safe answer; random with a 'zero-match' preference (guess words that share no letters with earlier guesses) is a cheap improvement.

FOLLOW-UP 3 The word list is much larger.

Computing all pairwise match counts is O(W^2 L). Sample candidates when choosing a guess, or precompute buckets by letter at each position to count matches faster.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find secret word with match-count
# feedback
#
# ## Problem You are given a list of **unique** lowercase
# words, all of the same length (e.g., length = 6). One of
# these words is a **secret** word. You can interact with
# a black-box API: - matchCount(guess) -> int: when you
# submit a word guess from the list, the API returns the
# number of positions i such that guess[i] == secret[i]
# (i.e., exact character match at the same index). You are
# allowed to call matchCount **at most 10 times**. ###
# Task Design/implement a function (e.g.,
# findSecretWord(words)) that uses the API to identify the
# secret word within the guess limit. ### Notes /
# Constraints - words contains between 1 and 100 words. -
# All words have equal length L (commonly L = 6). - Every
# guess must be a word from words. - After each API
# response, you may adapt your next guess.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Show how the candidate list shrinks.
# 2. Guess at random instead of the minimax choice.
# 3. The word list is much larger.
#
# INTERVIEW NOTES
# Each guess returns a number that every remaining
# candidate must agree with: a candidate can still be the
# secret only if it has the same match count with the
# guess as the secret does. So after each guess, filter
# the candidate list. To shrink fast, guess a word that
# splits the candidates evenly by match count (minimise
# the largest group); that is what keeps you inside 10
# guesses.
#
# BRUTE FORCE (say it first): Guess the first candidate
# each time, then filter
# Pick any remaining word, call the API, and keep only the
# words that have the same match count with it. Correct
# but there is no guarantee of finishing in 10 guesses on
# adversarial lists. Time O(guesses * W * L). Space O(W).


from collections import Counter
```

```
# BRUTE FORCE (say it first): Guess the first candidate
# each time, then filter | time O(guesses * W * L)
def _match(a, b):
    return sum(x == y for x, y in zip(a, b))


# HELPERS for the base question
def match(a, b):
    # number of positions holding the same letter
    return sum(x == y for x, y in zip(a, b))


def find_secret_first(words, secret):
    cand, guesses = list(words), 0
    while True:
        g = cand[0]
        guesses += 1
        m = _match(g, secret)
        if m == len(secret):
            return guesses
        cand = [w for w in cand if _match(g, w) == m]
```

```
# BASE QUESTION (optimal): Guess the word that minimises
# the size of its largest match-count group
# Idea: For each candidate guess count how many other
# candidates would fall into each match-count group, and
# pick the guess whose biggest group is smallest (a
# minimax choice).
def best_guess(candidates):
    # minimax: the guess whose worst answer leaves the
    # fewest
    # candidates (ties: alphabetical)
    def worst(g):
        sizes = Counter(match(g, w) for w in candidates)
        return max(sizes.values()), g
    return min(candidates, key=worst)


# BASE QUESTION (optimal): Guess the word that minimises
# the size of its largest match-count group
# Idea: For each candidate guess count how many other
# candidates would fall into each match-count group, and
# pick the guess whose biggest group is smallest (a
# minimax choice).
```

```
def find_secret(words, match_count, limit=10):
    # match_count(guess) is the black-box API
    candidates, guesses = list(words), 0
    while candidates and guesses < limit:
        guess = best_guess(candidates)
        guesses += 1
        score = match_count(guess)
        if score == len(guess):
            return guess, guesses
        # only words with the same score against this
        # guess
        # can still be the secret
        candidates = [w for w in candidates
                      if match(guess, w) == score]
    return None, guesses


# FOLLOW-UP 1: Show how the candidate list shrinks.
# Answer: After a guess with answer m, only words with the
# same match count against the guess survive (the secret
# and the candidate agree on every comparison).
def shrink_report(words, secret):
    # candidate counts after each guess (shows why it
    # works)
    sizes, candidates = [len(words)], list(words)
    while len(candidates) > 1:
        guess = best_guess(candidates)
        score = match(guess, secret)
        if score == len(secret):
            break
        candidates = [w for w in candidates
                      if match(guess, w) == score]
        sizes.append(len(candidates))
    return sizes


if __name__ == "__main__":
    import itertools
    words = ["".join(p) for p in
             itertools.product("abc", repeat=4)]
    worst = 0
    for secret in words:
        calls = []

        def api(guess, secret=secret):
```

```
            calls.append(guess)
            return match(guess, secret)
        found, used = find_secret(words, api)
        assert found == secret and used == len(calls)
        worst = max(worst, used)
    # 81 words are always found inside the limit of 10
    assert worst <= 10
    single = find_secret(["only"], lambda g: 4)
    assert single == ("only", 1)
    sizes = shrink_report(words, "abca")
    assert sizes[0] == 81
    assert sizes == sorted(sizes, reverse=True)
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], secret: str) -> str:
```

Sample call

```
solution(["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz
")
```

Test cases

Input	Expected
(["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz")	'acckzz'
(["secret"], "secret")	'secret'

STUDY HERE: topic 9, Greedy and Math

Site tags: algorithm-design, greedy, hard

HARD | 1Point3Acres | phone-screen, onsite-coding | 3 of 13

Wordle-Style 5-Letter Minimum-Guess Strategy

ASKED 2025-12-15 | 309 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 42.2** **30 DAYS 0** **PRIORITY 85/228**

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #843 Guess the Word

LeetCode match: LC 843 Guess the Word (related)

Frequency: Single report | Last asked: 2025-12-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/wordle-min-guess-strategy

QUESTION

Phone screen coding: given a black-box oracle that tells you whether a specific letter is at a specific position in a hidden 5-letter word, design an algorithm to identify the word in as few oracle calls as possible. Follow-up assumes a dictionary of candidate words is available.

Hidden word: a 5-letter string from a..z.
Oracle: query(pos: int, letter: char) -> bool returns whether position pos (0-indexed) holds letter.
Return the hidden word.
Primary metric: minimize the number of query calls.

Brute-force baseline: 5 × 26 = 130 calls.
Follow-up

A dictionary of candidate 5-letter words is available; design the strategy to use bigram/letter-frequency information to cut the expected number of calls.

EXAMPLES

```
HiddenWord = "GRAPE"
query(0, 'A') -> false
query(0, 'G') -> true   // first letter found
...
```

APPROACH

Drill information-theoretic puzzles (egg drop, twenty questions) to internalize the entropy-split heuristic.
Practice maintaining a candidate set under positive/negative oracle feedback (filter on each query result).
Be ready with a one-liner on Wordle's known optimal first guess (SOARE / CRANE) for intuition; do not hard-code it.

HOW TO THINK ABOUT IT

Each call returns one bit, so you can never beat ceil(log2(number of candidates)) calls. Without a dictionary the best you can do is order the questions by likelihood and stop at the first yes for each position (and skip the 26th letter, because it is implied). With a dictionary, keep the list of words still possible and ask the (position, letter) question whose yes/no answer splits that list most evenly.

Basis: Written for this question; related to LeetCode 843, Guess the Word, but the oracle answers one position and one letter per call.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
word = ""
for i in range(5):
    for c in "abcdefghijklmnopqrstuvwxyz":
        if query(i, c):
            word += c
            break
```

TIME	SPACE
130 calls	O(1)

WHY IT IS SLOW

Always 130 calls. Asking likely letters first, skipping the implied last letter, or splitting a dictionary of candidates in half needs far fewer.

OPTIMAL SOLUTION · Frequency order, or entropy split when a dictionary is known

TIME	SPACE
at most 5 x 25 calls with... at most 5 x 25 calls without a dictionary; about log2(words) with one	O(words)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What is the lower bound and when is it reached?

Every call returns at most one bit, so at least ceil(log2(candidates)) calls are needed. The entropy split reaches it only when a question can cut the candidates exactly in half; real dictionaries usually need a few more. A greedy scoring rule (closest to half) is enough to state; the interviewer mostly wants the entropy idea.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Wordle-Style 5-Letter Minimum-Guess
# Strategy
#
# Phone screen coding: given a black-box oracle that tells
# you whether a specific letter is at a specific position
# in a hidden 5-letter word, design an algorithm to
# identify the word in as few oracle calls as possible.
# Follow-up assumes a dictionary of candidate words is
# available. Hidden word: a 5-letter string from a..z.
# Oracle: query(pos: int, letter: char) -> bool returns
# whether position pos (0-indexed) holds letter. Return
# the hidden word. Primary metric: minimize the number of
# query calls. Brute-force baseline: 5 × 26 = 130 calls.
# Follow-up A dictionary of candidate 5-letter words is
# available; design the strategy to use bigram/letter-
# frequency information to cut the expected number of
# calls. Examples: HiddenWord = "GRAPE" query(0, 'A') ->
# false query(0, 'G') -> true // first letter found ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. What is the lower bound and when is it reached?
#
# INTERVIEW NOTES
# Each call returns one bit, so you can never beat
# ceil(log2(number of candidates)) calls. Without a
# dictionary the best you can do is order the questions by
# likelihood and stop at the first yes for each position
# (and skip the 26th letter, because it is implied). With
# a dictionary, keep the list of words still possible and
# ask the (position, letter) question whose yes/no answer
# splits that list most evenly.
#
# BRUTE FORCE (say it first): Ask every position and
# letter
# Query all 5 x 26 pairs and read the word off the
# answers. Always 130 calls, no matter how lucky the word
# is. Time 130 calls. Space O(1).


import string


# BRUTE FORCE: 130 calls
def brute_force(query):
    # 130 calls at most: every position, every letter
```

```
    return "".join(
        next(c for c in string.ascii_lowercase
             if query(i, c))
        for i in range(5))


# HELPERS for the base question
# letters from most to least common in English words
BY_FREQUENCY = "etaoinshrdlcumwfgypbvkjxqz"


# FOLLOW-UP 1: What is the lower bound and when is it
# reached?
# Answer: Every call returns at most one bit, so at least
# ceil(log2(candidates)) calls are needed. The entropy
# split reaches it only when a question can cut the
# candidates exactly in half; real dictionaries usually
# need a few more.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
```

```
def split_score(words, i, c):
    # how evenly does asking (position i, letter c) split?
    yes = sum(w[i] == c for w in words)
    return min(yes, len(words) - yes)


# BASE QUESTION (optimal): Frequency order, or entropy
# split when a dictionary is known
# Idea: Without a dictionary: for each position ask
# letters from most to least common and stop at the first
# yes; if 25 letters said no the 26th is implied and needs
# no call.
def by_frequency(query):
    # no dictionary: common letters first, stop at the
    # first
    # yes, and skip the 26th letter (it must be the one
    # left)
    calls, word = 0, []
    for i in range(5):
        for k, c in enumerate(BY_FREQUENCY):
            if k == 25:
                word.append(c)
                break
            calls += 1
            if query(i, c):
```

```
                word.append(c)
                break
    return "".join(word), calls


# BASE QUESTION (optimal): Frequency order, or entropy
# split when a dictionary is known
# Idea: Without a dictionary: for each position ask
# letters from most to least common and stop at the first
# yes; if 25 letters said no the 26th is implied and needs
# no call.
def with_dictionary(query, dictionary):
    # keep the words that still fit every answer so far
    words, calls = list(dictionary), 0
    while len(words) > 1:
        options = {(i, w[i]) for w in words
                   for i in range(5)}
        # ask the question that cuts the list closest to
        # half
        i, c = max(options, key=lambda o: split_score(
            words, o[0], o[1]))
        calls += 1
        hit = query(i, c)
        words = [w for w in words if (w[i] == c) == hit]
    return words[0], calls


if __name__ == "__main__":
    def oracle(word):
        return lambda i, c: word[i] == c

    secret = "grape"
    assert brute_force(oracle(secret)) == secret
    word, calls = by_frequency(oracle(secret))
    assert word == secret and calls <= 125
    words = ["grape", "grade", "trade", "crane", "plane",
             "slate", "stare", "share"]
    for target in words:
        ask = oracle(target)
        found, used = with_dictionary(ask, words)
        assert found == target
        # about log2(8) = 3 questions suffice; allow a few
        # more
        assert used <= 6
    # lower bound: one bit per call, so ceil(log2(8)) = 3
```

```
# calls
assert min(with_dictionary(oracle(w), words)[1]
           for w in words) >= 3
```

NOTES FROM CANDIDATES

Without the dictionary, query positions in order; per position, query letters in descending frequency (e, t, a, o, i, n, ...) and stop as soon as one returns true.
With the dictionary, maintain a candidate set; at each step pick the (position, letter) query whose true/false outcome splits the candidate set most evenly (entropy maximization).
Mention information theory: each query yields <= 1 bit, so the absolute lower bound on calls is ceil(log2(|candidates|)).
Don't waste minutes on a closed-form optimum — the interviewer mostly wants you to articulate the entropy heuristic and pick one reasonable scoring function.

PREPARATION

Drill information-theoretic puzzles (egg drop, twenty questions) to internalize the entropy-split heuristic.
Practice maintaining a candidate set under positive/negative oracle feedback (filter on each query result).
Be ready with a one-liner on Wordle's known optimal first guess (SOARE / CRANE) for intuition; do not hard-code it.

STUDY HERE: topic 9, Greedy and Math

HARD | PracHub | Technical Screen | 4 of 13

Shuffle array using random integer API

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 14.7** **30 DAYS 0** **PRIORITY 162/228**

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: the interview site's own solution

LeetCode match: LC 384 Shuffle an Array

Posted on PracHub: 2025-12-08 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/shuffle-array-using-random-integer-api

QUESTION

You are given an array **nums** of length **n** representing a deck of distinct cards. You **do not** have access to a built-in shuffle function, but you are given the following API:

```
rand_int(l, r) # returns a uniformly random integer in
 [l, r], inclusive
```

Task

Implement a function **`shuffle(nums)`** that randomly permutes the array **in place** such that **every one of the `n!` permutations is equally likely**.

Requirements

- You may only use **`rand_int(l, r)`** as your source of randomness.
- The algorithm should run in **`O(n)`** time and **`O(1)`** extra space.
- You may assume **`rand_int`** itself is perfectly uniform and independent between calls. Describe your algorithm, argue why the resulting permutation is uniform, and give the time and space complexity.

APPROACH

Fisher-Yates: for i from n-1 down to 1, j = rand_int(0, i), swap nums[i], nums[j]. Uniform because each permutation has probability 1/n!. Avoid the biased 'swap with random index in [0,n)' version.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether rand_int(l, r) is inclusive of r, whether to shuffle in place, and whether the result must be uniformly random over all n! permutations.

BRUTE FORCE TO SAY OUT LOUD

Assign each element a random key and sort. It is uniform but O(n log n).

OPTIMAL IDEA

Fisher-Yates: for i from n - 1 down to 1, pick j = rand_int(0, i) and swap nums[i] with nums[j]. Each of the n! permutations has probability 1 / n! because position i is filled by one of i + 1 equally likely elements. O(n) time, O(1) extra space.

DRY-RUN THESE CASES

n = 0 and n = 1. n = 3 with many trials: all six permutations should appear about equally. Say why swapping with a random index in [0, n) is biased (n^n outcomes cannot divide evenly into n! permutations).

LIKELY FOLLOW-UPS

Shuffle only k elements (stop the loop early), uniform integer from a coin flip only (rejection sampling), or a weighted shuffle.

PITFALLS

Using j in [0, n) every time. An off-by-one on the inclusive bound.

HOW TO THINK ABOUT IT

'Shuffle' means every one of the n! orders must be equally likely. Count outcomes: a procedure with n^n equally likely random choices cannot give n! equal buckets, so it is biased. Fisher-Yates makes n choices with sizes n, n-1, ..., 1, which multiply to exactly n!.

Basis: LeetCode 384, Shuffle an Array (Fisher-Yates).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def shuffle(nums):
    keyed = []
    for x in nums:
        keyed.append((rand_int(0, 10 ** 9), x))
    keyed.sort()
    return [x for _, x in keyed]
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

O(n log n) time and O(n) extra space, and equal random keys make the result very slightly non-uniform. Fisher-Yates (swap position i with a random position in 0..i) is O(n), in place and exactly uniform.

OPTIMAL SOLUTION · Fisher-Yates

TIME	SPACE
O(n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only k random elements.

Run Fisher-Yates for k steps from the front (swap position i with a random index in i..n-1) and return the first k. O(k) after copying, still uniform.

FOLLOW-UP 2 Only a coin-flip API exists, no rand_int.

Build a uniform integer in [0, m) by rejection sampling: draw ceil(log2 m) random bits and retry if the number is at least m. That keeps every value equally likely, unlike taking a remainder.

FOLLOW-UP 3 Why is the biased version biased?

It makes n choices of n options, n^n equally likely paths, and n! does not divide n^n for n >= 3, so some permutations must appear more often than others.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Shuffle array using random integer API
#
# You are given an array nums of length n representing a
# deck of distinct cards. You **do not** have access to a
# built-in shuffle function, but you are given the
# following API: python rand_int(l, r) # returns a
# uniformly random integer in [l, r], inclusive ### Task
# Implement a function shuffle(nums) that randomly
# permutes the array **in place** such that **every one of
# the n! permutations is equally likely**. ###
# Requirements - You may only use rand_int(l, r) as your
# source of randomness. - The algorithm should run in O(n)
# time and O(1) extra space. - You may assume rand_int
# itself is perfectly uniform and independent between
# calls. Describe your algorithm, argue why the resulting
# permutation is uniform, and give the time and space
# complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only k random elements.
# 2. Only a coin-flip API exists, no rand_int.
# 3. Why is the biased version biased?
#
# INTERVIEW NOTES
# 'Shuffle' means every one of the n! orders must be
# equally likely. Count outcomes: a procedure with n^n
# equally likely random choices cannot give n! equal
# buckets, so it is biased. Fisher-Yates makes n choices
# with sizes n, n-1, ..., 1, which multiply to exactly n!.
#
# BRUTE FORCE (say it first): Give each element a random
# key and sort
# Pair each element with a random number and sort by it.
# With keys from a large range it is practically uniform,
# but it spends O(n log n) and needs extra space; if two
# keys collide the result is slightly biased. Time O(n log
# n). Space O(n).


import random


# BRUTE FORCE (say it first): Give each element a random
# key and sort | time O(n log n)
def rand_int(lo, hi):
    return random.randint(lo, hi)


# HELPERS for the base question
def shuffle_by_keys(nums):
    keyed = sorted(
        (
            (rand_int(0, 10**9), i)
            for i in range(len(nums))
        )
    )
    return [nums[i] for _, i in keyed]


# BASE QUESTION (optimal): Fisher-Yates
# Idea: Walk i from n - 1 down to 1, pick j uniformly from
# 0..i (inclusive) and swap nums[i] with nums[j].
def shuffle(nums, rand_int):
    # Fisher-Yates: from the end, swap each position with
    # a
    # random one at or before it. All n! orders are
    # equally
    # likely because each step picks uniformly among i +
    # 1.
    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]
    return nums


# FOLLOW-UP 1: Return only k random elements.
# Answer: Run Fisher-Yates for k steps from the front
# (swap position i with a random index in i..n-1) and
# return the first k. O(k) after copying, still uniform.
def sample_k(nums, k, rand_int):
    # k random elements: only k Fisher-Yates steps, from
    # the front
    a = list(nums)
    for i in range(min(k, len(a))):
        j = rand_int(i, len(a) - 1)
        a[i], a[j] = a[j], a[i]
    return a[:k]


# FOLLOW-UP 2: Only a coin-flip API exists, no rand_int.
# Answer: Build a uniform integer in [0, m) by rejection
# sampling: draw ceil(log2 m) random bits and retry if the
# number is at least m. That keeps every value equally
# likely, unlike taking a remainder.
def rand_below(m, flip):
    # only a coin flip exists: rejection sampling gives a
    # uniform number in [0, m) (a remainder would be
    # biased)
    bits = max(1, (m - 1).bit_length())
    while True:
        x = 0
        for _ in range(bits):
            x = x * 2 + flip()
        if x < m:
            return x


# FOLLOW-UP 3: Why is the biased version biased?
# Answer: It makes n choices of n options, n^n equally
# likely paths, and n! does not divide n^n for n >= 3, so
# some permutations must appear more often than others.
def biased_shuffle(nums, rand_int):
    # the common WRONG version: n^n paths cannot be spread
```

```
    # over
    # n! orders evenly (shown only to explain the bias)
    for i in range(len(nums)):
        j = rand_int(0, len(nums) - 1)
        nums[i], nums[j] = nums[j], nums[i]
    return nums


if __name__ == "__main__":
    rng = random.Random(7)
    rand_int = lambda lo, hi: rng.randint(lo, hi)
    seen = {}
    for _ in range(6000):
        key = tuple(shuffle([1, 2, 3], rand_int))
        seen[key] = seen.get(key, 0) + 1
    # all 6 orders appear, each about 1000 times
    assert len(seen) == 6
    assert all(850 < v < 1150 for v in seen.values())
    # nothing is lost or repeated
    deck = shuffle(list(range(50)), rand_int)
    assert sorted(deck) == list(range(50))
    # k random elements
```

```
    pick = sample_k(list(range(10)), 3, rand_int)
    assert len(set(pick)) == 3 and len(pick) == 3
    # a coin gives uniform numbers below 5
    coin = lambda: rng.randint(0, 1)
    counts = [0] * 5
    for _ in range(5000):
        counts[rand_below(5, coin)] += 1
    assert all(800 < c < 1200 for c in counts)
    assert shuffle([], rand_int) == []
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int], random_indices: list[int]) -
> list[int]:
```

Sample call

```
solution([1, 2, 3, 4], [1, 1, 0])
```

Test cases

Input	Expected
([1, 2, 3, 4], [1, 1, 0])	[3, 1, 4, 2]
([], [])	[]

STUDY HERE: topic 9, Greedy and Math

HARD | PracHub | Technical Screen | 5 of 13

Find maximum value in huge text file

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 185/228**

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

LeetCode match: No LC match (systems / scalability question; the Hard tag looks inflated)

Posted on PracHub: 2025-12-08 (updated 2026-06-27)

Original: https://prachub.com/interview-questions/find-maximum-value-in-huge-text-file

QUESTION

You are given the path to a **very large** text file on disk. The file has the following properties:

- Each line contains exactly one signed 64-bit integer (in decimal).
- The file size can be tens or hundreds of gigabytes, so it **cannot** be fully loaded into memory.
- You have standard file I/O APIs that allow you to read the file sequentially (e.g., line by line or in chunks).

Tasks

1.
Single-machine solution Design an algorithm to compute the **maximum integer value** in this file.

- Assume you have limited RAM (much smaller than the file size).
- Discuss how you would read the file and track the result.
- Give high-level pseudocode.
- Analyze time and space complexity. 2.

Performance improvement / parallelization Suppose you now have a multi-core machine or multiple machines available and want to speed up the computation.

- How would you modify your approach to take advantage of parallelism (e.g., processing the file in chunks)?
- Explain how you would combine partial results to get the global maximum.
- Mention any edge cases or failure scenarios you would consider (e.g., very large values, empty file). Assume you do not have a database; you are working directly with the file system and your programming language's standard I/O libraries.

APPROACH

Stream the file line by line (or in large buffered chunks) keeping a single running max initialized to None/-2^63; O(n) time, O(1) memory. Parallelize by splitting byte ranges aligned to line boundaries across cores/machines (map), then take the max of the partial maxima (reduce). Edge cases: empty file, trailing newline, malformed lines, negative-only values, retries when a worker fails.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the file size relative to RAM, whether a line can be malformed, whether the file can be empty, and whether there is one machine or a cluster.

BRUTE FORCE TO SAY OUT LOUD

Load the file and call max. Say why it fails (memory), then move on.

OPTIMAL IDEA

Stream line by line with one running maximum initialized to None (not zero, so negative-only files work). O(n) time, O(1) memory. To parallelize split the file into byte ranges aligned to line boundaries: each worker discards its partial first line (the previous worker reads it) and finishes the line that crosses its end. Reduce with max of the partial results; retry failed ranges since each task is idempotent.

DRY-RUN THESE CASES

Empty file, one line, all negative values, last line without newline, values at the 64-bit limits.

LIKELY FOLLOW-UPS

Top K values, the median of a huge file, or distributed execution with failures.

PITFALLS

Starting the max at zero. Splitting ranges mid-line.

HOW TO THINK ABOUT IT

The answer is a running maximum, which needs one variable, so memory should not depend on the file size. Read the file as a stream. To go faster across cores, give each worker a byte range that starts and ends on line boundaries; the global maximum is the maximum of the partial maxima.

Basis: Systems / streaming question, no LeetCode match: a streaming reduce and its map-reduce parallelization.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(path):
    numbers = []
    for line in open(path):
        numbers.append(int(line))  # in memory
    return max(numbers)
```

BRUTE FORCE

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Needs memory for the whole file, which is exactly what is not available at tens of gigabytes. Streaming with one running maximum uses O(1) memory.

OPTIMAL SOLUTION · Single machine: stream the file, keep one running maximum

TIME	SPACE
O(n)	O(1) plus the read buffer

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Parallel: split byte ranges on line boundaries (map), max of the maxima (reduce)

TIME	SPACE
O(n / workers) plus I/O limits	O(workers)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Edge cases to mention.

Empty file (return None or raise, by contract); a final line without a newline; blank lines; malformed lines; values at the 64-bit extremes; an all-negative file (do not initialize the max to 0); and Windows line endings. Decide whether a bad line fails the job or is skipped and counted.

FOLLOW-UP 2 How do you scale to many machines and handle failures?

Assign byte ranges (map tasks) to workers through a coordinator, and make each task idempotent (it only returns its maximum), so a failed or slow worker is simply retried or speculatively re-executed elsewhere. The reduce step is a max over a tiny list. On a distributed filesystem, align ranges to its block size for locality. **Cost:** O(n / workers) wall-clock time plus O(workers) to combine; total work stays O(n).

FOLLOW-UP 3 Is it I/O-bound or CPU-bound, and why does that matter?

Parsing integers is cheap compared with disk throughput, so one sequential reader often saturates the disk and extra threads help only if the data sits on several disks or on fast SSD or network storage. Measure first; the parallel version pays off mainly across machines or storage devices. **Cost:** reading takes about n / disk throughput and parsing is O(n) as well, so extra threads help only beyond the disk's speed.

FOLLOW-UP 4 The file has billions of lines and one machine is too slow.

Split by byte offsets across machines (each skips to the next newline), compute local maxima, and take the maximum of those few numbers. Tasks are idempotent, so a failed worker can simply be retried. **Cost:** O(n / machines) per machine and O(machines) to combine; a retry re-runs only the failed byte range.

FOLLOW-UP 5 Now return the K largest values instead of only the maximum.

Keep a min-heap of size K while streaming: push a value if the heap is not full, or replace the root when the value is larger. Memory O(K), time O(n log K); merge the per-worker heaps for a parallel version.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find maximum value in huge text file
#
# You are given the path to a **very large** text file on
# disk. The file has the following properties: - Each line
# contains exactly one signed 64-bit integer (in decimal).
# - The file size can be tens or hundreds of gigabytes, so
# it **cannot** be fully loaded into memory. - You have
# standard file I/O APIs that allow you to read the file
# sequentially (e.g., line by line or in chunks). ###
# Tasks 1. **Single-machine solution** Design an algorithm
# to compute the **maximum integer value** in this file. -
# Assume you have limited RAM (much smaller than the file
# size). - Discuss how you would read the file and track
# the result. - Give high-level pseudocode. - Analyze time
# and space complexity. 2. **Performance improvement /
# parallelization** Suppose you now have a multi-core
# machine or multiple machines available and want to speed
# up the computation.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Edge cases to mention.
# 2. How do you scale to many machines and handle
# failures?
# 3. Is it I/O-bound or CPU-bound, and why does that
# matter?
# 4. The file has billions of lines and one machine is too
# slow.
# 5. Now return the K largest values instead of only the
```

```
# maximum.
#
# INTERVIEW NOTES
# The answer is a running maximum, which needs one
# variable, so memory should not depend on the file size.
# Read the file as a stream. To go faster across cores,
# give each worker a byte range that starts and ends on
# line boundaries; the global maximum is the maximum of
# the partial maxima.
#
# BRUTE FORCE (say it first): Load the whole file and take
# max
# Read every number into a list and call max. Correct for
# small files and the oracle; it needs memory proportional
# to the file, which is the thing the problem rules out.
# Time O(n). Space O(n).
```

```
import heapq
import os
import tempfile


# BRUTE FORCE (say it first): Load the whole file and take
# max | time O(n)
def max_in_file_loaded(path):
    with open(path) as f:
        nums = [int(x) for x in f.read().split()]
    return max(nums) if nums else None


# BASE QUESTION (optimal): Single machine: stream the
# file, keep one running maximum
# Idea: Never load the file. Read it through a buffered
# iterator and keep one variable, the maximum so far,
# started at None so that an all-negative file or a one-
# line file is handled correctly (do not start at 0).
def max_in_file(path):
    # one sequential pass, one number in memory
    best = None
    with open(path, "r", buffering=1 << 20) as f:
        for line in f:
            line = line.strip()
            # blank lines are ignored (state this
            # contract)
            if not line:
                continue
            value = int(line)
            # start from None, not 0: all numbers may be
            # negative
            if best is None or value > best:
                best = value
    return best


# ONE WORKER: its byte range
def max_in_range(path, start, end):
    # one worker: lines that START inside [start, end)
    best = None
    with open(path, "rb") as f:
        f.seek(start)
        if start:
```

```
            # we may be in the middle of a line: the
            # previous
            # worker owns it, so skip to the next line
            # start
            f.seek(start - 1)
            f.readline()
        while f.tell() < end:
            line = f.readline()
            if not line:
                break
            if line.strip():
                v = int(line)
                best = v if best is None else max(best, v)
    return best


# FOLLOW-UP 4: The file has billions of lines and one
# machine is too slow.
# Answer: Split by byte offsets across machines (each
# skips to the next newline), compute local maxima, and
# take the maximum of those few numbers. Tasks are
# idempotent, so a failed worker can simply be retried.
def parallel_max(path, workers=4):
    # split by byte offsets; combine the few local maxima
    size = os.path.getsize(path)
    cuts = [size * i // workers
            for i in range(workers + 1)]
    parts = [max_in_range(path, a, b)
             for a, b in zip(cuts, cuts[1:])]
    parts = [p for p in parts if p is not None]
    return max(parts) if parts else None


# FOLLOW-UP 5: Now return the K largest values instead of
# only the maximum.
# Answer: Keep a min-heap of size K while streaming: push
# a value if the heap is not full, or replace the root
# when the value is larger. Memory O(K), time O(n log K);
# merge the per-worker heaps for a parallel version.
def top_k_in_file(path, k):
    # the k largest values: a min-heap of size k, O(k)
    # memory
    heap = []
    with open(path) as f:
        for line in f:
```

```
            if not line.strip():
                continue
            value = int(line)
            if len(heap) < k:
                heapq.heappush(heap, value)
            elif value > heap[0]:
                heapq.heapreplace(heap, value)
    return sorted(heap, reverse=True)


if __name__ == "__main__":
    def write(lines):
        f = tempfile.NamedTemporaryFile("w", delete=False)
        f.write("".join(lines))
        f.close()
        return f.name

    p = write(["5\n", "-3\n", "\n", "99\n", "12"])
    assert max_in_file(p) == 99
    assert top_k_in_file(p, 2) == [99, 12]
    assert parallel_max(p, 3) == 99
    # all negative numbers, extreme 64-bit values
    q = write(["-7\n", "-2\n", "-9\n"])
    assert max_in_file(q) == -2
    big = write([f"{2 ** 63 - 1}\n", f"{-2 ** 63}\n"])
    assert max_in_file(big) == 2 ** 63 - 1
    # an empty file has no maximum
    e = write([])
    assert max_in_file(e) is None
    assert parallel_max(e) is None
    # every worker count agrees on a bigger file
    nums = [(i * 7919) % 10007 - 5000
            for i in range(2000)]
    r = write([f"{x}\n" for x in nums])
    for w in (1, 2, 5, 8):
        assert parallel_max(r, w) == max(nums)
    for name in (p, q, big, e, r):
```

```
os.remove(name)
```

STUDY HERE: topic 9, Greedy and Math

Site tags: algorithm-design, math-reasoning, hard

HARD | 1Point3Acres | onsite-coding | 6 of 13

Find All Bad Pairs Using runTest Oracle

ASKED 2025-11-27 | 327 DAYS AGO | OLDER

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 192/228**

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2025-11-27 | Roles: SWE, RS

Original: https://www.1point3acres.com/interview/problems/company/google/find-bad-pairs-with-oracle

QUESTION

PhD intern coding round 2: given a black-box oracle **`runTest(S)`** that returns **`true`** iff a subset **`S`** contains no bad pair, identify all bad pairs across N test items with as few oracle calls as possible.

Items t_1, ..., t_N are given.
Oracle: runTest(S: Set[Item]) -> bool returns true iff S contains zero bad pairs; otherwise false.
Output: every bad pair (t_i, t_j).
Minimize total oracle calls.

EXAMPLES

```
N=4, bad pairs = {(1,2)}.
runTest({1,2,3,4}) -> false -> at least one bad pair
    somewhere.
runTest({1,3,4})   -> true  -> 2 is in every bad pair
    containing items from this set.
... binary search to isolate.
```

APPROACH

Read the Wikipedia article on group testing and Dorfman's two-stage scheme.
Drill recursive binary partitioning on a candidate set with positive/negative oracle feedback.
Have a back-of-envelope analysis ready: "Total calls O(K log N) for K bad pairs and N items."

HOW TO THINK ABOUT IT

Testing every pair costs N(N-1)/2 calls. A single call on a whole set answers 'is there any bad pair inside', so you can discard good sets fast and binary-search for the culprit inside a bad one. Add the tests one at a time to groups known to be clean: if a group plus the new test fails, binary search the group for a partner; otherwise the new test joins it.

Basis: Adaptive group testing; the same method as the 'bad pairs with runTest' question in the PracHub set.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
bad = []
for i in range(n):
    for j in range(i + 1, n):
        if not runTest({i, j}):
            bad.append((i, j))
```

TIME	SPACE
O(N^2) calls	**O(1)**

WHY IT IS SLOW

Always N(N - 1) / 2 calls. Testing whole groups and binary searching a failing group finds each bad pair in about log N calls.

OPTIMAL SOLUTION · Group the tests and binary search for partners

TIME	SPACE
O(K log N + N * G) about calls (K bad pairs, G groups)	**O(N)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many calls are needed in the best case and the worst case?

With no bad pairs it takes N - 1 calls, one per test after the first. With K bad pairs each one costs about log N calls to locate, so roughly K log N plus the cost of building clean groups. The brute force is always N(N-1)/2, which is far larger as soon as bad pairs are rare.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find All Bad Pairs Using runTest Oracle
#
# PhD intern coding round 2: given a black-box oracle
# runTest(S) that returns true iff a subset S contains no
# bad pair, identify all bad pairs across N test items
# with as few oracle calls as possible. Items t_1, ...,
# t_N are given. Oracle: runTest(S: Set[Item]) -> bool
# returns true iff S contains zero bad pairs; otherwise
# false. Output: every bad pair (t_i, t_j). Minimize total
# oracle calls. Examples: N=4, bad pairs = {(1,2)}.
# runTest({1,2,3,4}) -> false -> at least one bad pair
# somewhere. runTest({1,3,4}) -> true -> 2 is in every bad
# pair containing items from this set. ... binary search
# to isolate.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many calls are needed in the best case and the
# worst case?
```

```
#
# INTERVIEW NOTES
# Testing every pair costs N(N-1)/2 calls. A single call
# on a whole set answers 'is there any bad pair inside',
# so you can discard good sets fast and binary-search for
# the culprit inside a bad one. Add the tests one at a
# time to groups known to be clean: if a group plus the
# new test fails, binary search the group for a partner;
# otherwise the new test joins it.
#
# BRUTE FORCE (say it first): Call runTest on every pair
# Test every pair of items with runTest and keep the
# failing pairs. Always N(N-1)/2 calls. Time O(N^2) calls.
# Space O(1).


import itertools


# BRUTE FORCE: ask about every pair
def bad_pairs_brute(items, run_test):
    # every pair costs a call: N * (N - 1) / 2 calls
    pairs = itertools.combinations(items, 2)
    return {(a, b) for a, b in pairs
            if not run_test({a, b})}
```

```
# HELPERS for the base question
def first_partner(group, t, run_test):
    # binary search: the shortest prefix of the group that
    # fails together with t; its last member is a partner
    lo, hi = 1, len(group)
    while lo < hi:
        mid = (lo + hi) // 2
        if run_test(set(group[:mid]) | {t}):
            lo = mid + 1
        else:
            hi = mid
    return group[lo - 1]


def partners(group, t, run_test):
    # all members of the group that are bad with t
    rest, found = list(group), []
    while rest and not run_test(set(rest) | {t}):
        x = first_partner(rest, t, run_test)
        found.append(x)
        # take it out and test the others again
        rest.remove(x)
    return found


# BASE QUESTION (optimal): Group the tests and binary
# search for partners
# Idea: Keep groups of tests that pass together.
def find_bad_pairs(items, run_test):
    # groups of items that pass together (no bad pair
    # inside)
    groups, pairs = [], set()
    for t in items:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                pairs.add((min(x, t), max(x, t)))
            # t joins the first group with no partner for
            # it
            if not found and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs


if __name__ == "__main__":
    def make_oracle(bad):
        calls = [0]

        def run_test(subset):
            calls[0] += 1
            ordered = sorted(subset)
            pairs = itertools.combinations(ordered, 2)
            return not any(p in bad for p in pairs)
        return run_test, calls

    bad = {(3, 17), (5, 29)}
    run, calls = make_oracle(bad)
    assert find_bad_pairs(list(range(30)), run) == bad
    # far fewer than the 435 calls of the brute force
    assert calls[0] < 435 // 2
    # no bad pairs: about one call per item
    run, calls = make_oracle(set())
    assert find_bad_pairs(list(range(30)), run) == set()
    assert calls[0] <= 30
    run, _ = make_oracle({(1, 2)})
    assert bad_pairs_brute([1, 2, 3, 4], run) == {(1, 2)}
```

NOTES FROM CANDIDATES

Standard "group testing" / Dorfman testing problem. Walk through the analogy with COVID pooled tests for intuition. Algorithm sketch:

Test the full set. If true, no bad pairs.
Otherwise, binary-search to find one bad pair by partitioning the set and testing halves.
Remove the items that participate in the located bad pair; repeat.

Tighter bound: with up to K bad pairs, total queries are O(K log N) using adaptive group testing.
A naive O(N²) pairwise scheme works but the interviewer expects you to beat it with adaptive splitting.
Verbalize correctness: each binary-search step rules out half the candidates; you can always reduce the search space until exactly one bad pair is isolated.

PREPARATION

Read the Wikipedia article on group testing and Dorfman's two-stage scheme.
Drill recursive binary partitioning on a candidate set with positive/negative oracle feedback.
Have a back-of-envelope analysis ready: "Total calls O(K log N) for K bad pairs and N items."

STUDY HERE: topic 9, Greedy and Math

HARD | IN APP LC 523 | Exponent | 7 of 13

Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TALK ALL TIME 25.5 30 DAYS 0 PRIORITY 135/228

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: your app's interview approach (LeetCode #523)

LeetCode match: LC 523 Continuous Subarray Sum

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | +610 interviews

Original: https://www.aced.io/questions/2765/subarray-sum-multiple-of-k

QUESTION

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6. The array [5, 0, 0, 0] with k = 3 also has a "good subarray" ([0, 0]), since the sum is 0, and 0 is a multiple of 3. Examples nums = [23, 2, 4, 7], k = 6 output: true nums = [5, 0, 0, 0], k = 3 output: true nums = null, k = 1 output: false In these examples, the output is true if there exists a "good subarray" in the given array nums for the specified k, and false otherwise.Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(n)?

APPROACH

Prefix sum mod k with a hash map of first-seen index; a repeat remainder at distance >= 2 means a valid subarray. Handle k = 0 and the remainder-0 case with {0: -1}. O(n).

HOW TO THINK ABOUT IT

A subarray sum is a difference of two prefix sums, and a multiple of k means the two prefix sums have the same remainder mod k. So remember the first index at which each remainder appeared; seeing it again at least two positions later gives a subarray of length at least two. Seed the table with remainder 0 at index -1 for subarrays that start at the beginning.

HOW TO THINK ABOUT IT

Basis: LeetCode 523, Continuous Subarray Sum: a subarray of at least two elements whose sum is a multiple of k. Assumed edge cases: nums null or shorter than 2 gives false; k = 0 means the sum must be 0.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, k):
    for i in range(len(nums)):
        total = nums[i]
        for j in range(i + 1, len(nums)):
            total += nums[j]
            if total % k == 0:
                return True
    return False
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

O(n^2) pairs. Two prefix sums with the same remainder mod k bound a subarray whose sum is a multiple of k, so a dictionary of first-seen remainders is O(n).

OPTIMAL SOLUTION · Prefix sums mod k with the first index of each remainder

TIME	SPACE
O(n)	O(min(n, k))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Count the subarrays whose sum is a multiple of k (any length, LeetCode 974).

Count how many times each remainder has occurred and add that count each time the remainder repeats (Python's % is already non-negative for negative sums). O(n).

FOLLOW-UP 2 Why store only the first index of each remainder?

A later occurrence of the same remainder would make the subarray shorter, never longer, so the first index is the best partner for the length-at-least-two condition.

FOLLOW-UP 3 The values can be negative or k huge.

Python's modulo gives a non-negative remainder; in languages where % can be negative normalise with ((s % k) + k) % k. A huge k just means the dictionary holds up to n entries. **Cost:** O(n) time with up to min(n, k) dictionary entries.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Given an integer array nums and an
# integer k, return true if nums has a subarray of at
# least two elements whose sum is a multiple of k.
#
# Determine if a given array contains a subarray of at
# least two elements whose sum is a multiple of a
# specified number k. An array is considered to have a
# "good subarray" if there exists at least one subarray
# (consisting of two or more elements) such that the sum
# of the elements in this subarray is a multiple of k. For
# example, the array [23, 2, 4, 7] with k = 6 has a "good
# subarray" ([2, 4]), as the sum 6 is a multiple of k = 6.
# The array [5, 0, 0, 0] with k = 3 also has a "good
# subarray" ([0, 0]), since the sum is 0, and 0 is a
# multiple of 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Count the subarrays whose sum is a multiple of k (any
# length, LeetCode 974).
# 2. Why store only the first index of each remainder?
# 3. The values can be negative or k huge.
#
# INTERVIEW NOTES
# A subarray sum is a difference of two prefix sums, and a
# multiple of k means the two prefix sums have the same
# remainder mod k. So remember the first index at which
# each remainder appeared; seeing it again at least two
# positions later gives a subarray of length at least two.
# Seed the table with remainder 0 at index -1 for
# subarrays that start at the beginning.
#
# BRUTE FORCE (say it first): Check every subarray of
# length at least two
# Extend each start with a running sum and test every end
# at least one element later. Correct and the oracle;
# quadratic. Time O(n^2). Space O(1).


# BRUTE FORCE (say it first, it also checks the answers)
def brute_good(nums, k):
    # oracle: try every block of at least two items
    for i in range(len(nums)):
        total = nums[i]
        for j in range(i + 1, len(nums)):
            total += nums[j]
            if (total % k == 0) if k else total == 0:
                return True
    return False


# BASE QUESTION (optimal): Prefix sums mod k with the
# first index of each remainder
# Idea: Walk the array keeping the running sum modulo k.
# If this remainder was seen before at index j and the
# current index i satisfies i - j >= 2, the elements
# between form a good subarray.
def has_good_subarray(nums, k):
    # prefix sums modulo k: two equal remainders at
    # indexes
    # at least 2 apart mean a block of 2 or more items
    # whose
    # sum is a multiple of k. Keep only the FIRST index of
    # each remainder: a later one only gives a shorter
    # block.
    first = {0: -1}
    total = 0
    for i, x in enumerate(nums):
        total += x
        # k == 0 means "equal to zero": compare the sums
        key = total % k if k else total
        if key not in first:
            first[key] = i
        elif i - first[key] >= 2:
            return True
    return False


# RETURN the subarray itself
def find_good_subarray(nums, k):
    # the same scan, but return the (start, end) indexes
    first = {0: -1}
    total = 0
    for i, x in enumerate(nums):
        total += x
        key = total % k if k else total
        if key not in first:
            first[key] = i
        elif i - first[key] >= 2:
            return first[key] + 1, i
    return None


# FOLLOW-UP 1: Count the subarrays whose sum is a multiple
# of k (any length, LeetCode 974).
# Answer: Count how many times each remainder has occurred
# and add that count each time the remainder repeats
# (Python's % is already non-negative for negative sums).
# O(n).
def count_multiples(nums, k):
    # follow-up (LeetCode 974): count the subarrays of any
    # length whose sum is a multiple of k. Python's % is
    # never
    # negative; elsewhere use ((s % k) + k) % k.
    seen = {0: 1}
    total = answer = 0
    for x in nums:
        total += x
        key = total % k
        # every earlier equal remainder is one more
        # subarray
        answer += seen.get(key, 0)
        seen[key] = seen.get(key, 0) + 1
    return answer


if __name__ == "__main__":
    assert has_good_subarray([23, 2, 4, 7], 6)
    assert has_good_subarray([23, 2, 6, 4, 7], 6)
    assert not has_good_subarray([23, 2, 6, 4, 7], 13)
    assert find_good_subarray([23, 2, 4, 7], 6) == (1, 2)
    # a single item is not enough, even for 0 and k
    assert not has_good_subarray([6], 6)
    assert has_good_subarray([0, 0], 5)
    assert not has_good_subarray([0, 1, 0], 0)
    assert has_good_subarray([0, 0], 0)
    assert has_good_subarray([5, -5], 7)
    assert has_good_subarray([1, 2], 10 ** 9) is False
    assert count_multiples([4, 5, 0, -2, -3, 1], 5) == 7
    import random
    for _ in range(500):
        nums = [random.randint(-6, 6)
                for _ in range(random.randint(0, 7))]
        k = random.randint(0, 6)
        want = brute_good(nums, k)
        assert has_good_subarray(nums, k) == want
```

OFFICIAL PAGE TEXT

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6. The array [5, 0, 0, 0] with k = 3 also has a "good subarray" ([0, 0]), since the sum is 0, and 0 is a multiple of 3. Examples

```
nums = [23, 2, 4, 7], k = 6
output: true

nums = [5, 0, 0, 0], k = 3
output: true

nums = null, k = 1
output: false
```

In these examples, the output is true if there exists a "good subarray" in the given array nums for the specified k, and false otherwise.Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(n)?Our solution processes the input array by iterating through it while maintaining a hash map (remainder_index_map) of remainders and their corresponding indices. For each element in the array, we accumulate the sum (sum_so_far) and compute the remainder when divided by k. We check if this remainder has been encountered before in the hash map. If the remainder is not in the hash map, we store the current index associated with this remainder. If the remainder has been seen before, we calculate the distance between the current index and the index of the previously seen remainder. If this distance is greater than or equal to the minimum subarray length (MIN_GOOD_SUBARRAY_LIMIT), we return True, indicating that a subarray with the required properties exists. If the loop completes without finding such a subarray, we return False. The use of a hash map ensures that each element is processed efficiently, as it allows constant-time lookups and insertions. Time Complexity: The solution has a time complexity of O(n), where n is the

OFFICIAL PAGE TEXT

number of elements in the array. This is because we iterate through the array once, performing constant-time operations (hash map lookups and insertions) for each element. Space Complexity: The solution uses O(n) space for the hash map that stores the remainders and their indices. In the worst case, all remainders are unique, resulting in the hash map containing n entries.Let's say you are given an integer array nums and an integer k. Return true if nums has a subarray where its length is at least two, and the sum of the elements in the subarray is a multiple of k. Watch Neeraj, a Senior Engineering Manager, give his answer to this software engineering interview question.No answers contributed by the community yet.Related questionsGiven an array of n integers and a number k, find the maximum number of elements whose sum is less than k, and return their sum.Given an array of integers and an integer k, find the number of continuous subarrays whose sum equals k.Given an array of integers and a number N, find the length of the longest contiguous subarray such that the difference between any two elements in the subarray is less than N.Related coursesCourseCourseMachine Learning Engineer Interview PrepLand your dream machine learning role at Meta, Google, Amazon, Apple, Microsoft, Nvidia, and other top companies. Learn from

mock interviews, frameworks, and advice from senior candidates. Explore ML system design, core concepts, coding, behavioral interviews, and more.Start learningCourseCourseData Science Interview PrepLand your dream data science role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Practice statistics, experimentation, coding, SQL, machine learning, behavioral interviews, and more.Start learningPythonJavaScript JavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

STUDY HERE: topic 9, Greedy and Math

HARD | IN APP LC 68 | Exponent | 8 of 13

★ TOP 75: rank 56 in the Medium and Hard priority list

★ Implement a function to justify a list of words within a given width.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TYPE **ALL TIME 44.7** **30 DAYS 0** **PRIORITY 79/228**

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: your app's interview approach (LeetCode #68)

LeetCode match: LC 68 Text Justification (inferred from short statement)

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/4903/function-justify-words-within-width

QUESTION

Implement a function to justify a list of words within a given width.

APPROACH

Greedily pack words per line; distribute spaces evenly (extra to the leftmost gaps), left-justify the last line and single-word lines. O(total chars).

HOW TO THINK ABOUT IT

Two decisions, kept separate: which words go on each line (pack greedily while the words plus one space between them fit), and how to spread the leftover spaces (an equal share in each gap, the extra spaces to the leftmost gaps). The last line and one-word lines are the special cases to name before coding.

Basis: LeetCode 68, Text Justification.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def justify(words, width):
    lines, i = [], 0
    while i < len(words):
        j = i + 1
        while j < len(words):
            if not fits(words[i:j + 1], width):
                break
            j += 1
        last = j == len(words)
        row = format_line(words[i:j],
                          width, last)
        lines.append(row)
        i = j
    return lines
```

BRUTE FORCE

TIME	SPACE
O(total characters * words per line)	O(width)

WHY IT IS SLOW

Every attempt to add a word re-joins the whole line to measure it, so each line costs about (words per line) times its length. Tracking only a running length makes it linear in the total characters.

OPTIMAL SOLUTION · Greedy line packing

TIME	SPACE
O(total characters)	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you change it to fully justify the last line too, or to center the text?

Only the final-line branch changes. For full justification drop the special case for the last line (it would be spread like every other line). For centering, split the leftover spaces between the two sides, giving the extra space to the right. The greedy packing step stays identical.

FOLLOW-UP 2 What if the words contain wide characters or different pixel widths?

Replace len(word) by a measure(word) function (for example font metrics or East Asian width) and compare against the pixel width. Packing and distribution logic stay the same; spaces are measured in the same unit, so the spare width may not divide evenly and the remainder must be assigned to gaps in whole space units.

FOLLOW-UP 3 Fully justify the last line too.

Drop the special case for the last line (a line with more than one word is spread like the others). Single-word lines stay left-aligned.

FOLLOW-UP 4 Words have pixel widths (a proportional font).

Replace len(word) with a measure function and compare against the pixel width; the extra space is distributed in whole space units, so the remainder rule stays the same.

FOLLOW-UP 5 Minimise raggedness instead of packing greedily.

That is the Knuth-Plass style DP: cost of a line = (spare space)^2, minimise the total over all ways to break the text. O(n * words per line).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement a function to justify a list of
# words within a given width.
#
# Implement a function to justify a list of words within a
# given width.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you change it to fully justify the last
# line too, or to center the text?
# 2. What if the words contain wide characters or
# different pixel widths?
# 3. Fully justify the last line too.
# 4. Words have pixel widths (a proportional font).
# 5. Minimise raggedness instead of packing greedily.
#
# INTERVIEW NOTES
# Two decisions, kept separate: which words go on each
# line (pack greedily while the words plus one space
# between them fit), and how to spread the leftover spaces
# (an equal share in each gap, the extra spaces to the
# leftmost gaps). The last line and one-word lines are the
# special cases to name before coding.
#
# BRUTE FORCE (say it first): Build each line by trying to
# add one more word with join
# Grow a line word by word, testing the joined length each
# time, then distribute the spaces. It does the same
# packing as the optimal version but recomputes lengths
# from scratch, which costs extra time. Time O(total
# characters * words per line). Space O(width).


# BRUTE FORCE (say it first): Build each line by trying to
# add one more word with join | time O(total characters *
# words per line)
def justify_naive(words, W):
    lines, i = [], 0
    while i < len(words):
        j = i + 1
        while (
            j < len(words)
            and len(" ".join(words[i : j + 1])) <= W
        ):
            j += 1
        line_words = words[i:j]
        if j == len(words) or len(line_words) == 1:
            lines.append(" ".join(line_words).ljust(W))
        else:
            letters = sum(len(w) for w in line_words)
            gaps = len(line_words) - 1
            sp, extra = divmod(W - letters, gaps)
            lines.append(
                "".join(
                    w
                    + " "
                    * (sp + (1 if k < extra else 0))
                    for k, w in enumerate(
                        line_words[:-1]
                    )
                )
                + line_words[-1]
            )
        i = j
    return lines


# HELPERS for the base question
def left_align(line, width, measure=len):
    text = " ".join(line)
    return text + " " * (width - measure(text))


def center(line, width, measure=len):
    # leftover spaces split on both sides, the odd one
    # right
    text = " ".join(line)
    spare = width - measure(text)
    left = spare // 2
    return " " * left + text + " " * (spare - left)


# FOLLOW-UP 2: What if the words contain wide characters
# or different pixel widths?
# Answer: Replace len(word) by a measure(word) function
# (for example font metrics or East Asian width) and
# compare against the pixel width.
# FOLLOW-UP 4: Words have pixel widths (a proportional
# font).
# Answer: Replace len(word) with a measure function and
# compare against the pixel width; the extra space is
# distributed in whole space units, so the remainder rule
# stays the same.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def pack_greedy(words, width, measure=len):
    # put as many words as fit on every line
    lines, line, used = [], [], 0
    for w in words:
        size = measure(w)
        # `used` counts the words and one space between
        # each
        if line and used + 1 + size > width:
            lines.append(line)
            line, used = [], 0
        used += size + (1 if line else 0)
        line.append(w)
    if line:
        lines.append(line)
    return lines


# FOLLOW-UP 2: What if the words contain wide characters
# or different pixel widths?
# Answer: Replace len(word) by a measure(word) function
# (for example font metrics or East Asian width) and
# compare against the pixel width.
# FOLLOW-UP 4: Words have pixel widths (a proportional
# font).
# Answer: Replace len(word) with a measure function and
# compare against the pixel width; the extra space is
# distributed in whole space units, so the remainder rule
# stays the same.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def spread(line, width, measure=len):
    # fully justified: spaces between the words, extra
    # ones
    # go to the LEFT gaps first. One word stays left-
```

```
    # aligned.
    if len(line) == 1:
        return line[0] + " " * (width - measure(line[0]))
    spaces = width - sum(measure(w) for w in line)
    gaps = len(line) - 1
    each, extra = divmod(spaces, gaps)
    out = ""
```

```
    for i, w in enumerate(line[:-1]):
        out += w + " " * (each + (1 if i < extra else 0))
    return out + line[-1]


# BASE QUESTION (optimal): Greedy line packing
# Idea: Pack as many words as fit on a line (counting one
# space between words). Spread the spare spaces over the
# gaps, giving the extra ones to the leftmost gaps.
# FOLLOW-UP 1: How would you change it to fully justify
# the last line too, or to center the text?
# Answer: Only the final-line branch changes. For full
# justification drop the special case for the last line
# (it would be spread like every other line).
# FOLLOW-UP 3: Fully justify the last line too.
# Answer: Drop the special case for the last line (a line
# with more than one word is spread like the others).
# Single-word lines stay left-aligned.
def justify(words, width, last="left", measure=len,
            lines=None):
    # last: "left" (the classic rule), "full" (spread it
    # too)
    # or "center". `lines` lets the caller bring its own
    # line breaks (see pack_optimal).
    lines = lines or pack_greedy(words, width, measure)
    out = []
    for i, line in enumerate(lines):
        if i < len(lines) - 1:
            out.append(spread(line, width, measure))
        # FOLLOW-UP 1, 3: justify the last line too
        elif last == "full":
            out.append(spread(line, width, measure))
        # FOLLOW-UP 1: centre the last line
        elif last == "center":
            out.append(center(line, width, measure))
        else:
            out.append(left_align(line, width, measure))
    return out


# FOLLOW-UP 5: Minimise raggedness instead of packing
# greedily.
# Answer: That is the Knuth-Plass style DP: cost of a line
# = (spare space)^2, minimise the total over all ways to
# break the text. O(n * words per line).
```

```
def pack_optimal(words, width):
    # follow-up: choose line breaks that minimise the
    # total
    # raggedness (Knuth-Plass idea). best[i] is the
    # cheapest
    # way to set the first i words.
    n = len(words)
    best = [0] + [float("inf")] * n
    prev = [0] * (n + 1)
    for i in range(1, n + 1):
        size = -1
        # a line holds words j..i-1, try the longest first
        for j in range(i - 1, -1, -1):
            size += len(words[j]) + 1
            if size > width:
                break
            cost = 0 if i == n else (width - size) ** 2
            if best[j] + cost < best[i]:
                best[i], prev[i] = best[j] + cost, j
    lines, i = [], n
    while i > 0:
        lines.append(words[prev[i]:i])
        i = prev[i]
    return lines[::-1]


# HELPERS for the checks below
def raggedness(lines, width):
    # sum of the squares of the unused space (last line
    # free)
    cost = 0
    for line in lines[:-1]:
        cost += (width - len(" ".join(line))) ** 2
    return cost


if __name__ == "__main__":
    text = ("This is an example of text justification."
            ).split()
    got = justify(text, 16)
    assert got == ["This    is    an", "example  of text",
                   "justification.  "]
    words = "What must be acknowledgment shall be".split()
    got = justify(words, 16)
    assert got == ["What   must   be", "acknowledgment  ",
                   "shall be        "]
    assert all(len(line) == 16 for line in got)
    full = justify(words, 16, last="full")
    assert full[2] == "shall         be"
    mid = justify(words, 16, last="center")
    assert mid[2] == "    shall be    "
    assert justify(["a"], 3) == ["a  "]
    assert justify([], 5) == []
    # pixel widths: every character is 2 units wide
    wide = justify(["ab", "cd", "ef"], 12,
                   measure=lambda s: 2 * len(s))
    assert wide == ["ab    cd", "ef" + " " * 8]
    import random
    for _ in range(200):
        ws = ["x" * random.randint(1, 6)
              for _ in range(random.randint(1, 9))]
        width = 8
        greedy = pack_greedy(ws, width)
        best = pack_optimal(ws, width)
        assert sum(best, []) == ws
        fits = [len(" ".join(ln)) for ln in best]
        assert max(fits) <= width
        assert raggedness(best, width) <= raggedness(
            greedy, width)
```

STUDY HERE: topic 9, Greedy and Math

MEDIUM | PracHub | Technical Screen | 9 of 13

Find the Largest Monotone Increasing Number

POSTED 2026-07-04 | 108 DAYS AGO | LAST 6 MONTHS

STUDY: TALK ALL TIME 5 30 DAYS 0 PRIORITY 171/228

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: the interview site's own solution

LeetCode match: LC 738 Monotone Increasing Digits (inferred from overview)

Posted on PracHub: 2026-07-04 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/find-the-largest-monotone-increasing-number

QUESTION

Overview: Find the largest number no greater than n whose decimal digits are monotone increasing. Develop a linear-time greedy correction that revisits earlier decreasing boundaries and fills the remaining suffix optimally without introducing leading zeros.

APPROACH

Scan digits right to left; when digit[i-1] > digit[i], decrement digit[i-1] and mark position i as the point to fill with 9s. O(d).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether monotone means non-decreasing (equal digits allowed) or strictly increasing, whether n can be 0, and whether n is given as an integer or a very long string.

BRUTE FORCE TO SAY OUT LOUD

Decrement n until its digits are monotone. Correct but up to O(n * d).

OPTIMAL IDEA

Scan the digits from right to left. When digit[i - 1] > digit[i], decrease digit[i - 1] by one and remember i as the first position to fill with 9s. After the scan set every digit from that marker to the end to 9. O(d) time.

DRY-RUN THESE CASES

10 gives 9, 332 gives 299, 1234 stays 1234, 100 gives 99, 0 gives 0, 1000 gives 999.

LIKELY FOLLOW-UPS

Strictly increasing digits variant, or return the largest number below n when n itself is already valid.

PITFALLS

Scanning left to right, which misses the cascading decrement. Setting 9s from the wrong index.

HOW TO THINK ABOUT IT

If a digit is smaller than the one before it, the earlier digit has to drop by one and everything after that point can become 9 (as large as possible). Dropping a digit can break the pair before it, so scan from the right: each repair is checked against the digit to its left. That is why one right-to-left pass is enough.

Basis: LeetCode 738, Monotone Increasing Digits (non-decreasing digits).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    while not is_non_decreasing(str(n)):
        n -= 1
    return n
```

TIME	SPACE
O(n * d)	O(1)

WHY IT IS SLOW

n can be 10^9, so it may take a billion steps. A right-to-left scan that lowers a digit where order breaks and fills the rest with 9s is O(digits).

OPTIMAL SOLUTION · Right-to-left scan, then fill with nines

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The number is given as a string with 100,000 digits.

The scan works directly on the digit list, so keep it as a list of characters and never convert to an integer until the end (or not at all). The algorithm is O(d).

FOLLOW-UP 2 Return the smallest monotone number that is at least n instead.

Scan left to right for the first digit that is larger than its right neighbour and copy that digit over the whole suffix (332 becomes 333). The prefix before it is already monotone.

FOLLOW-UP 3 Digits must be strictly increasing.

Same idea, but the suffix to fill is the largest strictly increasing run (for example ...6789), and you must check that the prefix leaves enough room. Say the extra care and write the check as a brute-force comparison first.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find the Largest Monotone Increasing
# Number
#
# Find the largest number no greater than n whose decimal
# digits are monotone increasing. Develop a linear-time
# greedy correction that revisits earlier decreasing
# boundaries and fills the remaining suffix optimally
# without introducing leading zeros.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The number is given as a string with 100,000 digits.
# 2. Return the smallest monotone number that is at least
# n instead.
# 3. Digits must be strictly increasing.
#
# INTERVIEW NOTES
# If a digit is smaller than the one before it, the
# earlier digit has to drop by one and everything after
# that point can become 9 (as large as possible). Dropping
# a digit can break the pair before it, so scan from the
# right: each repair is checked against the digit to its
# left. That is why one right-to-left pass is enough.
#
# BRUTE FORCE (say it first): Decrement n until its digits
# are monotone
# Starting at n, subtract one until every digit is at
# least the one before it. Obviously correct, but n can be
# 10^9, so it can take a billion steps. Time O(n * d).
# Space O(1).


# BRUTE FORCE (say it first): Decrement n until its digits
# are monotone | time O(n * d)
def _is_monotone(n):
    s = str(n)
    return all(a <= b for a, b in zip(s, s[1:]))


def monotone_brute(n):
    while not _is_monotone(n):
        n -= 1
    return n


# BASE QUESTION (optimal): Right-to-left scan, then fill
```

```
# with nines
# Idea: Walk i from the last digit to the first. When
# digit[i - 1] > digit[i], decrease digit[i - 1] by one
# and remember i as the start of the nines.
def largest_monotone(n):
    # largest number <= n whose digits never decrease
    d = list(str(n))
    mark = len(d)
    # scan from the right; a drop forces the left digit
    # down by
    # one and everything after it up to 9
    for i in range(len(d) - 1, 0, -1):
        if d[i - 1] > d[i]:
            d[i - 1] = str(int(d[i - 1]) - 1)
            mark = i
    for i in range(mark, len(d)):
        d[i] = "9"
    return int("".join(d))
```

```
# FOLLOW-UP 1: The number is given as a string with
# 100,000 digits.
# Answer: The scan works directly on the digit list, so
# keep it as a list of characters and never convert to an
# integer until the end (or not at all). The algorithm is
# O(d).
def largest_monotone_string(digits):
    # the same on a string of any length (no big integers)
    d = list(digits)
    mark = len(d)
    for i in range(len(d) - 1, 0, -1):
        if d[i - 1] > d[i]:
            d[i - 1] = chr(ord(d[i - 1]) - 1)
            mark = i
    d[mark:] = "9" * (len(d) - mark)
    # a leading zero (from 10 -> 09) is dropped
    return "".join(d).lstrip("0") or "0"


# FOLLOW-UP 2: Return the smallest monotone number that is
# at least n instead.
# Answer: Scan left to right for the first digit that is
# larger than its right neighbour and copy that digit over
# the whole suffix (332 becomes 333). The prefix before it
# is already monotone.
```

```
def smallest_monotone_at_least(n):
    # follow-up: smallest monotone number >= n
    d = list(str(n))
    for i in range(len(d) - 1):
        if d[i] > d[i + 1]:
            # copy the bigger digit over the rest: 332 ->
            # 333
            d[i + 1:] = d[i] * (len(d) - i - 1)
            break
    return int("".join(d))


# HELPERS for the checks below
def is_monotone(n):
    s = str(n)
    return all(a <= b for a, b in zip(s, s[1:]))


if __name__ == "__main__":
    assert largest_monotone(10) == 9
    assert largest_monotone(1234) == 1234
    assert largest_monotone(332) == 299
    assert largest_monotone(0) == 0
    assert largest_monotone_string("332") == "299"
    assert largest_monotone_string("10") == "9"
    assert smallest_monotone_at_least(332) == 333
    assert smallest_monotone_at_least(120) == 122
    # check against brute force for many numbers
    for n in range(0, 3000):
        every = range(n + 1)
        want = max(x for x in every if is_monotone(x))
        assert largest_monotone(n) == want
        text = largest_monotone_string(str(n))
        assert text == str(want)
        later = range(n, 99999)
        up = next(x for x in later if is_monotone(x))
        assert smallest_monotone_at_least(n) == up
    # a 100,000 digit input is fine as a string
    big = "9" * 50000 + "1" * 50000
    assert largest_monotone_string(big)[:3] == "899"
```

FUNCTION AND TEST CASES

Sample call

```
largest_monotone_increasing(0)
```

Test cases

Input	Expected
`(0,)`	`0`
`(9,)`	`9`

STUDY HERE: topic 9, Greedy and Math

MEDIUM | PracHub | Technical Screen | 10 of 13

Min boards to cover roof holes

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK ALL TIME 19.6 30 DAYS 0 PRIORITY 150/228

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #452 Minimum Number of Arrows to Burst Balloons

LeetCode match: No direct match; related LC 1326 Minimum Number of Taps / LC 452 (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-05)

Original:
https://prachub.com/coding-questions/min-boards-to-cover-roof-holes

QUESTION

Overview: This question evaluates competency in combinatorial optimization and problem modeling for coverage constraints, testing algorithmic reasoning, complexity analysis, and efficient representation of grid-based data.

APPROACH

Greedy interval cover: sort hole positions, place a board starting at the first uncovered hole, skip all holes within its length, repeat.

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy has only the overview, so ask: are holes points on a line or cells in a grid, what is the board length, can boards be placed in either orientation, and must each board be axis aligned. The 1D case is a classic interval cover.

BRUTE FORCE TO SAY OUT LOUD

Try every placement of boards. Exponential.

OPTIMAL IDEA

1D with boards of length L: sort the holes, place a board starting at the first uncovered hole so it covers [h, h + L - 1], skip every hole inside it, repeat. The count is the answer, and the exchange argument shows starting at the leftmost uncovered hole is optimal. O(n log n). For a grid with horizontal boards only, do this per row. If boards can be placed in either orientation covering rows or columns, it becomes a minimum vertex cover in a bipartite graph (Konig's theorem), solved with matching.

DRY-RUN THESE CASES

No holes (0), one hole, holes spread more than L apart, adjacent holes, holes at the same position.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Boards of different lengths, or a limit on the number of boards.

PITFALLS

Starting a board in the middle of an uncovered gap. Not sorting the holes first.

HOW TO THINK ABOUT IT

Cover the leftmost hole first: any board that covers it can be slid right until it starts exactly at that hole without uncovering anything to its right, so start a board at the leftmost uncovered hole and skip everything it covers. Repeating that is optimal because starting later never covers more.

Basis: Written; interval covering (compare LeetCode 452 and 1326). Assumed rule (confirm it): holes are integer positions on a line, a board of length L covers positions p .. p + L - 1, boards may be placed anywhere; minimise the number of boards.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = [INF] * (len(holes) + 1)
best[0] = 0
for i in range(1, len(holes) + 1):
    for j in range(i):
        span = holes[i - 1] - holes[j] + 1
        if span <= L:  # one board covers j..i-1
            best[i] = min(best[i], best[j] + 1)
answer = best[len(holes)]
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every i tries every j, O(n^2). Starting a board at the leftmost uncovered hole and skipping everything it covers is optimal and O(n log n) including the sort.

OPTIMAL SOLUTION · Greedy: start a board at the leftmost uncovered hole

TIME	SPACE
O(n log n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You have only K boards: what is the smallest board length that covers all holes?

Binary search the length L and test with the greedy (is the number of boards at most K?); feasibility is monotone in L. O(n log range).

FOLLOW-UP 2 Boards of different lengths with a cost each.

The greedy exchange argument breaks. Use DP over the sorted holes: best[i] = min over board types of best[j] + cost, where j is the first hole not covered by that board starting at hole i.

FOLLOW-UP 3 Holes are in a grid and boards are 1 x L horizontally.

Apply the greedy independently on every row. If boards may also be vertical it becomes a covering problem on a bipartite graph (König's theorem).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Min boards to cover roof holes
#
# This question evaluates competency in combinatorial
# optimization and problem modeling for coverage
# constraints, testing algorithmic reasoning, complexity
# analysis, and efficient representation of grid-based
# data.
#
# FOLLOW-UPS (answered below, in this order):
# 1. You have only K boards: what is the smallest board
# length that covers all holes?
# 2. Boards of different lengths with a cost each.
# 3. Holes are in a grid and boards are 1 x L
# horizontally.
#
# INTERVIEW NOTES
# Cover the leftmost hole first: any board that covers it
# can be slid right until it starts exactly at that hole
# without uncovering anything to its right, so start a
# board at the leftmost uncovered hole and skip everything
# it covers. Repeating that is optimal because starting
# later never covers more.
#
# BRUTE FORCE (say it first): DP over the number of holes
# covered
# best[i] is the fewest boards that cover the first i
# holes: try every start j of the last board (it must
# cover holes j..i-1 within length L). Exact and the
# oracle; quadratic. Time O(n^2). Space O(n).


from bisect import bisect_left


# BRUTE FORCE (say it first): DP over the number of holes
# covered | time O(n^2)
def boards_dp(holes, L):
    h = sorted(set(holes))
    best = [0] + [float("inf")] * len(h)
    for i in range(1, len(h) + 1):
        for j in range(i):
            if h[i - 1] - h[j] < L:
                best[i] = min(best[i], best[j] + 1)
    return best[-1]
```

```
# BASE QUESTION (optimal): Greedy: start a board at the
# leftmost uncovered hole
# Idea: Sort the holes. Place a board starting at the
# first uncovered hole, which covers every hole below
# start + L; skip those and repeat.
def min_boards(holes, length):
    # greedy: the leftmost uncovered hole starts a board,
    # which
    # then covers `length` positions
    count, covered_to = 0, float("-inf")
    for p in sorted(set(holes)):
        if p > covered_to:
            count += 1
            covered_to = p + length - 1
    return count


# FOLLOW-UP 1: You have only K boards: what is the
# smallest board length that covers all holes?
# Answer: Binary search the length L and test with the
# greedy (is the number of boards at most K?); feasibility
# is monotone in L. O(n log range).
def smallest_length(holes, k):
    # only k boards: the shortest board length that still
    # covers everything (more length never hurts: binary
    # search)
    pts = sorted(set(holes))
    if not pts:
        return 0
    lo, hi = 1, pts[-1] - pts[0] + 1
    while lo < hi:
        mid = (lo + hi) // 2
        if min_boards(pts, mid) <= k:
            hi = mid
        else:
            lo = mid + 1
    return lo


# FOLLOW-UP 2: Boards of different lengths with a cost
# each.
# Answer: The greedy exchange argument breaks. Use DP over
# the sorted holes: best[i] = min over board types of
# best[j] + cost, where j is the first hole not covered by
```

```
# that board starting at hole i.
def min_cost(holes, boards):
    # boards = [(length, cost)], any number of each: a DP
    # over
    # the sorted holes (the greedy no longer works)
    pts = sorted(set(holes))
    INF = float("inf")
    best = [0] * (len(pts) + 1)
    for i in range(len(pts) - 1, -1, -1):
        best[i] = INF
        for length, cost in boards:
            # the first hole this board does not reach
            j = bisect_left(pts, pts[i] + length)
            best[i] = min(best[i], cost + best[j])
    return best[0]


# FOLLOW-UP 3: Holes are in a grid and boards are 1 x L
# horizontally.
# Answer: Apply the greedy independently on every row. If
# boards may also be vertical it becomes a covering
# problem on a bipartite graph (König's theorem).
def boards_per_row(grid, length):
    # holes in a grid ('#'), horizontal boards: greedy per
    # row
    total = 0
    for row in grid:
        holes = [c for c in range(len(row))
                 if row[c] == "#"]
        total += min_boards(holes, length)
```

```
    return total


if __name__ == "__main__":
    assert min_boards([1, 2, 4, 5, 9], 3) == 3
    assert min_boards([], 3) == 0
    assert min_boards([5, 5, 5], 1) == 1
    # with 2 boards the shortest length is 5 (covers 1..5,
    # 9..13)
    assert smallest_length([1, 2, 4, 5, 9], 2) == 5
    assert smallest_length([1, 2, 4, 5, 9], 1) == 9
    assert smallest_length([], 1) == 0
    # a long board costs 5, two short ones cost 3 each
    assert min_cost([1, 2, 3, 10], [(3, 5), (1, 3)]) == 8
    assert min_cost([1, 2, 3, 4], [(2, 3), (4, 5)]) == 5
```

```
    roof = ["#.#.", "..##", "...."]
    assert boards_per_row(roof, 2) == 3
```

FUNCTION AND TEST CASES

Signature

```
def solution(roof: list[list[int]]) -> int:
SAMPLE INPUT
[[1,0,1],[0,1,1]]
```

Test cases

Input	Expected
[[1,0,1],[0,1,1]]	2
[[1,1],[1,1]]	0

STUDY HERE: topic 9, Greedy and Math

MEDIUM | PracHub | Technical Screen | 11 of 13

Choose optimal guesses for green-only Wordle

POSTED 2025-10-14 | 371 DAYS AGO | OLDER

STUDY: TALK ALL TIME 42.2 30 DAYS 0 PRIORITY 86/228

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #843 Guess the Word

LeetCode match: No direct match; related LC 843 Guess the Word (inferred from overview)

Posted on PracHub: 2025-10-14 (updated 2026-03-29)

Original: https://prachub.com/interview-questions/choose-optimal-guesses-for-green-only-wordle

QUESTION

Overview: This question evaluates probabilistic reasoning, expected-value optimization, information partitioning, and algorithmic efficiency in selecting guesses under partial (green-only) feedback; it belongs to the Coding & Algorithms domain, specifically search and probabilistic modeling, and emphasizes practical algorithm design and implementation rather than purely theoretical concepts. It is commonly asked to assess a candidate's ability to model uncertainty with weighted distributions, minimize the expected remaining search space, and manage time/space trade-offs for inputs up to several thousand words.

APPROACH

Filter candidates by green-position constraints, then pick the guess that maximizes expected information (partition sizes) over the remaining words.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what feedback exists (only green: letter in the right position), whether answers are equally likely or weighted, whether guesses must come from the candidate list, and the objective (minimize the expected size of the remaining set).

BRUTE FORCE TO SAY OUT LOUD

For each candidate guess and each possible answer compute the feedback and count the survivors. O(W^2 * L).

OPTIMAL IDEA

INTERVIEW-DAY PLAN

A guess g splits the candidates by its pattern against each answer, a bitmask of which positions are green. If groups have sizes s_1 ... s_k, the expected number of survivors is the sum of s_i^2 divided by W (weights replace sizes with weight sums). Pick the guess minimizing it. To speed up, store for each (position, letter) a bitset of candidate words so groups come from bit operations instead of pairwise comparisons.

DRY-RUN THESE CASES

Two candidates (any guess), words that differ in one position, a guess that is itself the answer (pattern all green), a single candidate.

LIKELY FOLLOW-UPS

Full Wordle feedback with yellow, multiple rounds, or an adaptive strategy.

PITFALLS

Minimizing the number of groups instead of the expected survivors. Recomputing the filter from scratch each round.

HOW TO THINK ABOUT IT

A guess splits the candidates into groups by which positions turned green. If the groups have sizes s_1 ... s_k out of W candidates, the expected size of what remains is the sum of s_i squared divided by W (a candidate falls in a group of size s with probability s / W, and then s remain). Pick the guess with the smallest value. To make it fast, store for each (position, letter) the set of words with that letter there as a bitset, so the groups come from a few AND operations.

Basis: Written. Assumed rules (confirm them): words of equal length, the answer is uniformly random among the candidates, a guess must be a candidate, feedback marks only the positions where the guess letter equals the answer letter (green). Goal: minimise the expected number of candidates left.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(guesses, answers):
    best, best_score = None, INF
    for g in guesses:
        groups = Counter()
        for a in answers:
            groups[green_pattern(g, a)] += 1
        sizes = groups.values()
        score = sum(c * c for c in sizes)
        score /= len(answers)
        if score < best_score:
            best, best_score = g, score
    return best
```

TIME	SPACE
O(W^2 * L)	O(W)

WHY IT IS SLOW

BRUTE FORCE

Compares every pair of words, O(W^2 * L). Bitsets per (position, letter) split the candidates into pattern groups with a few AND operations.

OPTIMAL SOLUTION · Bitsets per (position, letter), splitting groups with AND

TIME	SPACE
O(W * L * groups) with bit-parallel work per group	**O(W * L)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Play a whole game, using the feedback after each guess.

Keep the candidates that would have produced exactly the observed green pattern (filter_candidates), then choose the best guess among the remaining ones. Each round at least removes the guess itself, so the game ends.

FOLLOW-UP 2 Answers are not equally likely (weights).

Replace group sizes by group weights: score = sum(weight(group)^2) / total weight. The grouping step is unchanged; use sums of weights instead of popcounts.

FOLLOW-UP 3 Real Wordle also gives yellow feedback.

The pattern becomes three-valued per position (green, yellow, grey) with the usual duplicate-letter rules, so the pattern space is 3^L; bitsets still work if you precompute patterns, but the computation is O(W^2) pattern evaluations for a full table.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Choose optimal guesses for green-only
# Wordle
#
# This question evaluates probabilistic reasoning,
# expected-value optimization, information partitioning,
# and algorithmic efficiency in selecting guesses under
# partial (green-only) feedback; it belongs to the Coding
# & Algorithms domain, specifically search and
# probabilistic modeling, and emphasizes practical
# algorithm design and implementation rather than purely
# theoretical concepts. It is commonly asked to assess a
# candidate's ability to model uncertainty with weighted
# distributions, minimize the expected remaining search
# space, and manage time/space trade-offs for inputs up to
# several thousand words.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Play a whole game, using the feedback after each
# guess.
# 2. Answers are not equally likely (weights).
# 3. Real Wordle also gives yellow feedback.
#
# INTERVIEW NOTES
# A guess splits the candidates into groups by which
# positions turned green. If the groups have sizes s_1 ...
# s_k out of W candidates, the expected size of what
# remains is the sum of s_i squared divided by W (a
# candidate falls in a group of size s with probability s
# / W, and then s remain). Pick the guess with the
# smallest value. To make it fast, store for each
# (position, letter) the set of words with that letter
# there as a bitset, so the groups come from a few AND
# operations.
#
# BRUTE FORCE (say it first): Compute the feedback of
# every guess against every answer
# For each guess compute its green pattern against each
# candidate answer, count the sizes of the pattern groups,
# and evaluate sum(s^2) / W. Correct and the oracle; it
# compares every pair of words. Time O(W^2 * L). Space
# O(W).


from collections import Counter
from collections import defaultdict
```

```
from fractions import Fraction


# HELPERS for the brute force
def green(guess, answer):
    # green-only feedback: which positions match exactly
    return tuple(g == a for g, a in zip(guess, answer))


def partition(guess, words):
    # group the possible answers by the feedback they
    # would give
    groups = defaultdict(list)
    for w in words:
        groups[green(guess, w)].append(w)
    return list(groups.values())


def score(guess, words, weight=None):
    # expected number of answers left after this guess:
    # sum(group^2) / total, with optional per-word weights
    weight = weight or (lambda w: 1)
    total = sum(weight(w) for w in words)
    spread = sum(sum(weight(w) for w in g) ** 2
                 for g in partition(guess, words))
    return Fraction(spread, total)


# BRUTE FORCE (say it first): Compute the feedback of
# every guess against every answer | time O(W^2 * L)
def _green_mask(guess, answer):
    return tuple(g == a for g, a in zip(guess, answer))


def best_guess_brute(words):
    best = None
    for g in words:
        groups = Counter(
            _green_mask(g, a) for a in words
        )
        score = Fraction(
            sum(s * s for s in groups.values()),
            len(words),
        )
        if best is None or (score, g) < best:
```

```
            best = (score, g)
    return best[1]


# BASE QUESTION (optimal): Bitsets per (position, letter),
# splitting groups with AND
# Idea: Index the words: bits[pos][letter] is an integer
# whose bit i is set when word i has that letter at pos.
def best_guess(words, weight=None):
    # the guess that leaves the fewest answers on average
    # (ties: alphabetical, so the result is deterministic)
    def rank(g):
        return (score(g, words, weight), g)
    return min(words, key=rank)


# FASTER: bitsets
def best_guess_bitsets(words):
    # faster: one bitmask per (position, letter); each
    # position
    # splits every group in two with two AND operations
    W = len(words)
    bits = [defaultdict(int)
            for _ in range(len(words[0]))]
    for i, w in enumerate(words):
        for pos, ch in enumerate(w):
            bits[pos][ch] |= 1 << i
    best = None
    for g in words:
        groups = [(1 << W) - 1]
        for pos, ch in enumerate(g):
            mask, nxt = bits[pos][ch], []
            for grp in groups:
                for part in (grp & mask, grp & ~mask):
                    if part:
                        nxt.append(part)
            groups = nxt
        sizes = [bin(g).count("1") for g in groups]
        spread = sum(s * s for s in sizes)
        cand = (Fraction(spread, W), g)
        if best is None or cand < best:
            best = cand
    return best[1]
```

```
# FOLLOW-UP 1: Play a whole game, using the feedback after
# each guess.
# Answer: Keep the candidates that would have produced
# exactly the observed green pattern (filter_candidates),
# then choose the best guess among the remaining ones.
# Each round at least removes the guess itself, so the
# game ends.
def filter_candidates(words, guess, pattern):
    # keep only answers that would give the observed
    # feedback
    return [w for w in words
            if green(guess, w) == pattern]


# HELPERS for the checks below
def play(words, secret):
    # a whole game: guess, read the feedback, narrow down
    candidates, guesses = list(words), []
    while True:
        guess = best_guess(candidates)
        guesses.append(guess)
        if guess == secret:
            return guesses
        pattern = green(guess, secret)
        candidates = filter_candidates(
            candidates, guess, pattern)


if __name__ == "__main__":
```

```
    words = ["crane", "crate", "grape", "grade", "trade",
             "slate", "plate"]
    assert best_guess(words) == best_guess_bitsets(words)
    # a guess can never leave more than all the words
    assert score("crane", words) <= len(words)
    for secret in words:
        path = play(words, secret)
        assert path[-1] == secret
        # each round removes at least the guess itself
        assert len(path) <= len(words)
    # weights: a very likely word makes its guess more
    # attractive
    heavy = lambda w: 100 if w == "slate" else 1
    slate = score("slate", words, heavy)
    crane = score("crane", words, heavy)
    assert slate < crane
```

```
    assert partition("crane", ["crane"]) == [["crane"]]
```

FULL QUESTION TEXT FROM THE SITE

Problem

You are playing a simplified 5-letter word guessing game.

There is a hidden secret word of length 5 .
You have a dictionary of valid 5-letter words. Each word may also have an associated frequency/weight representing how likely it is to be the secret.
When you guess a 5-letter word, the game returns only “green” information (correct letter in the correct position). There is no information about letters that exist in the word but are in the wrong position (“yellow”), and no explicit info about letters that are absent.
Feedback definition

For a guess g and secret s, define the feedback as a 5-bit mask m where:

m[i] = 1 iff g[i] == s[i]
otherwise m[i] = 0

Example: g="crane", s="cared" -> mask is 1 0 0 0 0 (only position 0 matches).

Task

Given:

words : a list of N valid 5-letter words
weight[w] (optional): a positive number for each word (if omitted, assume uniform)
candidates : the current set of possible secret words consistent with feedback so far (initially all words)

Write an algorithm/function to choose the next guess that is optimal under the following objective:

Choose a guess word g (typically from words) that minimizes the expected size of the remaining candidate set after receiving the green-only mask, where the expectation is taken over the secret word drawn from candidates according to weight .

Formally, if C is the current candidate set and C_m(g) is the subset of candidates that would produce mask m for guess g, choose g that minimizes:

FULL QUESTION TEXT FROM THE SITE

E
[
|
C
m
a
s
k
(
g
)
|
]
=
sum
m
in
{
0
,
1
}
5
P
(

m
a
s
k
=
m
)
*
|
C
m
(
g
)
|
E[|C
mask

(g)|]=sum
min{0,1}
5

P(mask=m)*|C
m

(g)|

FULL QUESTION TEXT FROM THE SITE

where
P
(
m
a
s
k
=
m
)
P(mask=m) is induced by the weighted distribution over secrets in C.

Input/Output expectations
Input sizes: 1 <= N <= 5,000 (you may propose optimizations if needed), word length is always 5.
Output: the selected next guess word.
Follow-ups (optional)
How would you incorporate non-uniform word frequencies?
How would you scale if N is very large (e.g., 100k)?
How would you change the objective to minimize expected number of guesses instead of expected remaining candidates?

ALSO REPORTED · 1 more

Wordle-Style 5-Letter Minimum-Guess Strategy | 1Point3Acres | phone-screen, onsite-coding | asked 2025-12-15 | Single report

https://www.1point3acres.com/interview/problems/company/google/wordle-min-guess-strategy

Without the dictionary, query positions in order; per position, query letters in descending frequency (e, t, a, o, i, n, ...) and stop as soon as one returns true.
With the dictionary, maintain a candidate set; at each step pick the (position, letter) query whose true/false outcome splits the candidate set most evenly (entropy maximization).
Mention information theory: each query yields <= 1 bit, so the absolute lower bound on calls is ceil(log2(|candidates|)).
Don't waste minutes on a closed-form optimum — the interviewer mostly wants you to articulate the entropy heuristic and pick one reasonable scoring function.

Preparation: Drill information-theoretic puzzles (egg drop, twenty questions) to internalize the entropy-split heuristic.
Practice maintaining a candidate set under positive/negative oracle feedback (filter on each query result).
Be ready with a one-liner on Wordle's known optimal first guess (SOARE / CRANE) for intuition; do not hard-code it.

STUDY HERE: topic 9, Greedy and Math

MEDIUM | Exponent | 12 of 13

Write SQL code to publish the Fibonacci series.

ASKED (ANY CO.) ~4 MONTHS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 196/228**

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: written and tested here for this exact question

LeetCode match: No LC match (SQL) (inferred from short statement)

Last asked (any company): 4 months ago, about 2026-06 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/2779/sql-code-fibonacci-series

QUESTION

Write SQL code to publish the Fibonacci series.

APPROACH

Use a recursive CTE producing (n, a, b) rows and select a; limit the row count with a WHERE clause.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which database dialect, how many terms, and whether the series starts at 0 or 1.

BRUTE FORCE TO SAY OUT LOUD

A loop in a stored procedure. Works but is procedural.

OPTIMAL IDEA

Use a recursive common table expression holding (n, a, b): the anchor row is (1, 0, 1) and the recursive step is (n + 1, b, a + b) while n is below the limit. Select a. Watch integer overflow for large n; use a wider type or stop at the type limit.

DRY-RUN THESE CASES

Limit 1, limit 2, limit 10, a limit large enough to overflow a 64-bit integer.

LIKELY FOLLOW-UPS

Return only even terms or the sum of the first n terms.

PITFALLS

Forgetting the termination condition, which makes the query recurse until the database stops it.

HOW TO THINK ABOUT IT

A sequence that depends on the previous rows is a recursive CTE: one anchor row, one recursive step, and a stopping condition in the step. Carry the two previous values in each row (a, b) so the next row is (b, a + b); a row-per-term table is built in one statement with no procedural loop.

HOW TO THINK ABOUT IT

Basis: Written; recursive common table expressions. Run and checked on SQLite; the same query works on PostgreSQL and MySQL 8 (SQL Server omits the word RECURSIVE).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
a, b = 0, 1
for _ in range(n):
    execute("INSERT INTO fib VALUES (?)", a)
    a, b = b, a + b
```

TIME	SPACE
O(n)	**O(n)**
statements	

WHY IT IS SLOW

One statement and one round trip per term, and a loop outside the query language. A recursive CTE produces all rows in one set-based statement.

OPTIMAL SOLUTION · Recursive CTE carrying (n, a, b)

TIME	SPACE
O(n)	**O(1)**
	working set (O(n) result)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the even terms, or the sum of the first n terms.

Keep the CTE and filter or aggregate in the outer query: WHERE a % 2 = 0 for the even terms, SUM(a) for the total. The recursion stays the same.

FOLLOW-UP 2 Which database?

PostgreSQL and MySQL 8 use WITH RECURSIVE; SQL Server drops the word RECURSIVE and may need a CAST so the column type is wide enough (bigint) in the anchor row; Oracle accepts WITH without RECURSIVE too. State the dialect assumption first.

FOLLOW-UP 3 Large n overflows.

Use a wider type (BIGINT, NUMERIC, or DECIMAL) in the anchor row so the column type is wide enough from the start; arbitrary-size values need a language with big integers. **Cost:** O(n) rows regardless of the type; a wider type only costs more bytes per row.

FOLLOW-UP 4 Use it from application code with a parameter.

Bind the limit as a query parameter (never build the SQL by string concatenation) and fetch the single column. The recursion lives inside the database, so there is one round trip regardless of n.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Write SQL code to publish the Fibonacci
# series.
#
# Write SQL code to publish the Fibonacci series.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the even terms, or the sum of the first n terms.
# 2. Which database?
# 3. Large n overflows.
# 4. Use it from application code with a parameter.
#
# INTERVIEW NOTES
# A sequence that depends on the previous rows is a
# recursive CTE: one anchor row, one recursive step, and a
# stopping condition in the step. Carry the two previous
# values in each row (a, b) so the next row is (b, a + b);
# a row-per-term table is built in one statement with no
# procedural loop.
#
# BRUTE FORCE (say it first): Procedural loop that issues
# one statement per term
# Compute each term in application code (or a stored
# procedure) and INSERT it row by row. It works anywhere,
# but it needs a loop outside the query language and one
# round trip per term. Time O(n) statements. Space O(n).


import sqlite3


# HELPERS for the base question
# a recursive query: each row is computed from the
# previous one
FIB_SQL = """
WITH RECURSIVE fib(n, a, b) AS (
    SELECT 1, 0, 1
    UNION ALL
    SELECT n + 1, b, a + b FROM fib WHERE n < ?
)
SELECT a FROM fib
"""


# BASE QUESTION (optimal): Recursive CTE carrying (n, a,
# b)
```

```
# Idea: The anchor row is (1, 0, 1): the first term is 0
# and the next pair is (0, 1). Each recursive step
# produces (n + 1, b, a + b) while n is below the limit,
# so the term in column a is the Fibonacci number.
# FOLLOW-UP 4: Use it from application code with a
# parameter.
# Answer: Bind the limit as a query parameter (never build
# the SQL by string concatenation) and fetch the single
# column. The recursion lives inside the database, so
# there is one round trip regardless of n.
def fibonacci(count):
    # FOLLOW-UP 4: from app code: count is a parameter
    # one round trip: the recursion runs inside the
    # database;
    # the limit is a bound parameter, never glued into the
    # SQL
    with sqlite3.connect(":memory:") as db:
        rows = db.execute(FIB_SQL, (count,)).fetchall()
    return [r[0] for r in rows]


# CHECK in Python
def fibonacci_python(count):
    # the same sequence in Python, used to check the SQL
    a, b, out = 0, 1, []
    for _ in range(count):
        out.append(a)
        a, b = b, a + b
    return out


# FOLLOW-UP 1: Only the even terms, or the sum of the
# first n terms.
# Answer: Keep the CTE and filter or aggregate in the
# outer query: WHERE a % 2 = 0 for the even terms, SUM(a)
# for the total. The recursion stays the same.
def even_terms(count):
    # follow-up: filter in an outer query
    sql = f"SELECT a FROM ({FIB_SQL}) WHERE a % 2 = 0"
    with sqlite3.connect(":memory:") as db:
        return [r[0] for r in db.execute(sql, (count,))]


# FOLLOW-UP 1: Only the even terms, or the sum of the
# first n terms.
```

```
# Answer: Keep the CTE and filter or aggregate in the
# outer query: WHERE a % 2 = 0 for the even terms, SUM(a)
# for the total. The recursion stays the same.
def total(count):
    # follow-up: the sum of the first n terms
    sql = "SELECT SUM(a) FROM (" + FIB_SQL + ")"
    with sqlite3.connect(":memory:") as db:
        return db.execute(sql, (count,)).fetchone()[0]


if __name__ == "__main__":
    got = fibonacci(10)
    assert got == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
    assert fibonacci(10) == fibonacci_python(10)
    assert fibonacci(1) == [0]
    assert even_terms(10) == [0, 2, 8, 34]
    assert total(10) == sum(fibonacci_python(10))
    # 80 terms stay exact in a 64-bit column
    assert fibonacci(80) == fibonacci_python(80)
```

STUDY HERE: topic 9, Greedy and Math

MEDIUM | IN APP LC 465 | HackTheRounds | Onsite | 13 of 13

Trip Expense Settlement

STUDY: TALK ALL TIME 25.5 30 DAYS 0 PRIORITY 140/228

ABOUT THIS QUESTION

Topic: Greedy and Math

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #465 Optimal Account Balancing

LeetCode match: LC 465 (Optimal Account Balancing)

QUESTION

A group of friends go on a trip. Each friend may pay for others. Given payment records, compute minimal payments required to settle all debts.

EXAMPLES

Example 1

Input	`[{"payer": "alice", "amount": 4000, "payees": ["bob", "jess", "alice", "sam"]}, {"payer": "jess", "amount": 2000, "payees": ["jess", "alice"]}]`
Output	`[{"payer": "bob", "amount": 1000, "payee": "alice"}, {"payer": "sam", "amount": 1000, "payee": "alice"}]`
Why	`Calculate net balance for each person, then settle with minimum transactions.`

Example 2

Input	`expenses = [{"payer": "Alice", "amount": 40, "participants": ["Alice", "Bob"]}, {"payer": "Bob", "amount": 20, "participants": ["Alice", "Bob"]}]`
Output	`["Bob pays Alice 10.00"]`

CONSTRAINTS

- 1 <= payments.length <= 1000
- 1 <= amount <= 10^6
- 1 <= payees.length <= 100

APPROACH

Compute each person's net balance, then match debtors and creditors with heaps

HOW TO THINK ABOUT IT

First reduce the records to one number per person: net balance = paid - owed. Only that matters; who owed whom originally is irrelevant. Then settling is moving money between people with positive and negative balances. Matching largest creditor with largest debtor is quick and uses at most n - 1 payments, but only searching groups of people who sum to zero gives the true minimum.

Basis: LeetCode 465, Optimal Account Balancing. Each record is split equally among the listed payees (the payer may be one of them).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def settle(i):
    while i < n and net[i] == 0:
        i += 1  # already settled
    if i == n:
        return 0
    best = INF
    for j in range(i + 1, n):
        if net[j] * net[i] < 0:  # opposite sign
            net[j] += net[i]  # move balance i
            best = min(best, 1 + settle(i + 1))
            net[j] -= net[i]  # undo
    return best
```

TIME	SPACE
O(n!) worst case	O(n)

WHY IT IS SLOW

It tries every pairing, up to n! in the worst case. Matching the largest creditor with the largest debtor (heaps) is O(n log n) and uses at most n - 1 payments, though not always the minimum.

OPTIMAL SOLUTION · Heaps: match the largest debtor with the largest creditor

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do you need the true minimum number of payments?

That is NP-hard in general (it is the zero-sum partition problem). The greedy answer is at most n - 1 payments and is usually accepted; for exact answers use the bitmask DP above (about 20 people) or backtracking with pruning.

FOLLOW-UP 2 Amounts that do not divide evenly (cents).

Work in integer cents and decide where the remainder goes (to the payer, or round-robin among payees). Never use floats for money, and make the rule deterministic.

FOLLOW-UP 3 Different shares per person instead of equal split.

Replace the equal share with an explicit amount per payee in each record; the net balance per person is computed the same way.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Trip Expense Settlement
#
# A group of friends go on a trip. Each friend may pay for
# others. Given payment records, compute minimal payments
# required to settle all debts.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do you need the true minimum number of payments?
# 2. Amounts that do not divide evenly (cents).
# 3. Different shares per person instead of equal split.
#
# INTERVIEW NOTES
# First reduce the records to one number per person: net
# balance = paid - owed. Only that matters; who owed whom
# originally is irrelevant. Then settling is moving money
# between people with positive and negative balances.
# Matching largest creditor with largest debtor is quick
# and uses at most n - 1 payments, but only searching
# groups of people who sum to zero gives the true minimum.
#
# BRUTE FORCE (say it first): Backtracking over who pays
# whom (exact minimum)
# Take the first person with a nonzero balance and try
# settling them with every later person of the opposite
# sign, recursing on the rest; keep the fewest payments.
# This gives the exact minimum, but the search explodes
# with the number of people. Time O(n!) worst case. Space
# O(n).


import heapq
from itertools import combinations


# HELPERS for the brute force
def balances(records):
    # net balance per person: what they paid minus what
    # they
    # owe. Positive: the group owes them. All sum to zero.
    net = {}
    for rec in records:
        payer = rec["payer"]
        net[payer] = net.get(payer, 0) + rec["amount"]
        for person, share in shares_of(rec).items():
            net[person] = net.get(person, 0) - share
    return {p: v for p, v in net.items() if v}


# BRUTE FORCE (say it first): Backtracking over who pays
# whom (exact minimum) | time O(n!) worst case
def settle_min_backtracking(records):
    debts = list(balances(records).values())

    def go(i):
        while i < len(debts) and debts[i] == 0:
            i += 1
        if i == len(debts):
            return 0
        best = float("inf")
        for j in range(i + 1, len(debts)):
            if debts[i] * debts[j] < 0:
                debts[j] += debts[i]
                best = min(best, 1 + go(i + 1))
                debts[j] -= debts[i]
        return best

    return go(0)


# BASE QUESTION (optimal): Heaps: match the largest debtor
# with the largest creditor
# Idea: Push creditors (positive net) in a max-heap and
# debtors (negative net) in another. Repeatedly pay
# min(debt, credit) from the biggest debtor to the biggest
# creditor, and push back whoever still has a remainder.
def settle_greedy(records):
    # the biggest debtor pays the biggest creditor, as
    # much
    # as one side can: at most n - 1 payments
    net = balances(records)
    owed = [(-v, p) for p, v in net.items() if v > 0]
    debt = [(v, p) for p, v in net.items() if v < 0]
    heapq.heapify(owed)
    heapq.heapify(debt)
    out = []
    while owed and debt:
        o_amt, creditor = heapq.heappop(owed)
        d_amt, debtor = heapq.heappop(debt)
        pay = min(-o_amt, -d_amt)
        out.append({"payer": debtor, "amount": pay,
                    "payee": creditor})
        if -o_amt > pay:
            heapq.heappush(owed, (o_amt + pay, creditor))
        if -d_amt > pay:
            heapq.heappush(debt, (d_amt + pay, debtor))
    return out


# FOLLOW-UP 1: Do you need the true minimum number of
# payments?
# Answer: That is NP-hard in general (it is the zero-sum
# partition problem). The greedy answer is at most n - 1
# payments and is usually accepted; for exact answers use
# the bitmask DP above (about 20 people) or backtracking
# with pruning.
def min_payments_exact(values):
    # follow-up: the true minimum. People split into
    # groups
    # that settle among themselves (sum zero); a group of
    # g
    # people needs g - 1 payments. So find the most
    # groups:
    # a DP over subsets of people.
    n = len(values)
    total = [0] * (1 << n)
    best = [0] * (1 << n)
    for mask in range(1, 1 << n):
        low = (mask & -mask).bit_length() - 1
        rest = mask & (mask - 1)
        total[mask] = total[rest] + values[low]
```

```
        top = max((best[mask ^ (1 << i)]
                   for i in range(n) if mask >> i & 1))
        best[mask] = top + (1 if total[mask] == 0 else 0)
    return n - best[(1 << n) - 1]


# FOLLOW-UP 1: Do you need the true minimum number of
# payments?
# Answer: That is NP-hard in general (it is the zero-sum
# partition problem). The greedy answer is at most n - 1
# payments and is usually accepted; for exact answers use
# the bitmask DP above (about 20 people) or backtracking
# with pruning.
def min_payments_search(values):
    # the same answer by backtracking: settle the first
```

```
    # open
    # debt against every opposite one
    values = [v for v in values if v]

    def go(start):
        while start < len(values) and values[start] == 0:
            start += 1
        if start == len(values):
            return 0
        best = float("inf")
        seen = set()
        for j in range(start + 1, len(values)):
            if values[j] * values[start] < 0 and (
                    values[j] not in seen):
                seen.add(values[j])
                values[j] += values[start]
                best = min(best, 1 + go(start + 1))
                values[j] -= values[start]
        return best

    return go(0)


# FOLLOW-UP 2: Amounts that do not divide evenly (cents).
# Answer: Work in integer cents and decide where the
# remainder goes (to the payer, or round-robin among
# payees). Never use floats for money, and make the rule
# deterministic.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def split_equally(amount, people):
    # follow-up: cents that do not divide evenly. Work in
    # whole cents; the leftover cents go to the first
    # people
    # in sorted order, so the rule is always the same.
    people = sorted(people)
    base, extra = divmod(amount, len(people))
    return {p: base + (1 if i < extra else 0)
            for i, p in enumerate(people)}


# FOLLOW-UP 3: Different shares per person instead of
# equal split.
# Answer: Replace the equal share with an explicit amount
# per payee in each record; the net balance per person is
```

```
# computed the same way.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def shares_of(record):
    # follow-up: explicit shares per person instead of
    # equal
    payees = record["payees"]
    if isinstance(payees, dict):
        if sum(payees.values()) != record["amount"]:
            raise ValueError("shares do not add up")
        return payees
    return split_equally(record["amount"], payees)


# HELPERS for the checks below
def apply(payments, net):
    # check: after the payments every balance is zero
    left = dict(net)
    for pay in payments:
        who, whom = pay["payer"], pay["payee"]
        left[who] = left.get(who, 0) + pay["amount"]
        left[whom] = left.get(whom, 0) - pay["amount"]
    return all(v == 0 for v in left.values())


if __name__ == "__main__":
    trip = [{"payer": "alice", "amount": 4000,
             "payees": ["bob", "jess", "alice", "sam"]},
            {"payer": "jess", "amount": 2000,
             "payees": ["jess", "alice"]}]
    got = settle_greedy(trip)
    want = [
        {"payer": "bob", "amount": 1000,
         "payee": "alice"},
        {"payer": "sam", "amount": 1000,
         "payee": "alice"}]
    assert sorted(got, key=lambda p: p["payer"]) == want
    assert balances(trip) == {"alice": 2000, "bob": -1000,
                              "sam": -1000}
    # 10.00 split three ways: 3.34 + 3.33 + 3.33
    assert split_equally(1000, ["c", "a", "b"]) == {
        "a": 334, "b": 333, "c": 333}
    custom = [{"payer": "x", "amount": 90,
               "payees": {"x": 10, "y": 30, "z": 50}}]
    owed = {"x": 80, "y": -30, "z": -50}
```

```
    assert balances(custom) == owed
    assert min_payments_exact([5, -5]) == 1
    assert min_payments_exact([3, -3, 2, -2]) == 2
    assert min_payments_search([3, -3, 2, -2]) == 2
    import random
    for _ in range(100):
        names = "abcdef"
        records = []
        for _ in range(random.randint(1, 5)):
            count = random.randint(1, 4)
            who = random.sample(names, count)
            cents = random.randint(1, 500)
            records.append({"payer": random.choice(names),
                            "amount": cents,
                            "payees": who})
        net = balances(records)
        assert sum(net.values()) == 0
        pays = settle_greedy(records)
        assert apply(pays, net)
        assert len(pays) <= max(len(net) - 1, 0)
        values = list(net.values())
        exact = min_payments_exact(values)
        assert exact == min_payments_search(values)
        assert exact <= len(pays)
```

ALSO REPORTED · 1 more

Compute minimal transfers to settle group expenses | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimal-transfers-to-settle-group-expenses